

Sistemas Operativos // resumen final

> kernel & compilación

> threads (ULT/KLT)

> cgroups + namespaces

> protección & seguridad

> multiprocesadores

> módulos, drivers, syscalls

> virtualización

> docker + containers

> filesystems, RAID, LVM

> deadlocks

1. Kernel de Linux y compilación

Qué es el kernel

El **kernel** es la porción de código que reside en **memoria principal** y actúa como **intermediario entre el hardware y las aplicaciones**. Se ejecuta en modo privilegiado, ofrece a los procesos de usuario una interfaz controlada a través de **system calls** y es, en sentido estricto, el sistema operativo en sí mismo. El resto de piezas —shell, utilidades, librerías como `libc`— forman parte del “SO en sentido amplio” (Linux + GNU = GNU/Linux), pero el núcleo, lo mínimo indispensable, es el kernel.

Sus **cinco funciones principales** son la administración de la **memoria principal**, el manejo del uso de la **CPU** (scheduling), la administración de **procesos**, la gestión de **entrada/salida** y la coordinación de **comunicación y concurrencia** entre procesos. Ningún proceso de usuario puede acceder al hardware ni salirse de su propio espacio de direcciones sin pedírselo al kernel a través de una syscall.

Modos de ejecución

El hardware provee al menos dos **modos de ejecución**, señalizados por un **bit de modo** en la CPU:

- **Modo supervisor / kernel / privilegiado:** habilita el conjunto completo de instrucciones, incluyendo las privilegiadas (acceso a HW, cambio de tablas de páginas, deshabilitar interrupciones, etc.). Solo se ejecutan acá.
- **Modo usuario:** el proceso ve únicamente su propio espacio de direcciones y un subconjunto reducido de instrucciones.

La CPU arranca en modo supervisor y desciende a modo usuario cuando le cede el control a un proceso. La **única** forma de volver a modo kernel es a través de un **trap o interrupción** (una syscall es exactamente eso: una interrupción de software controlada). Un proceso no puede “decidir” cambiar de modo por su cuenta.

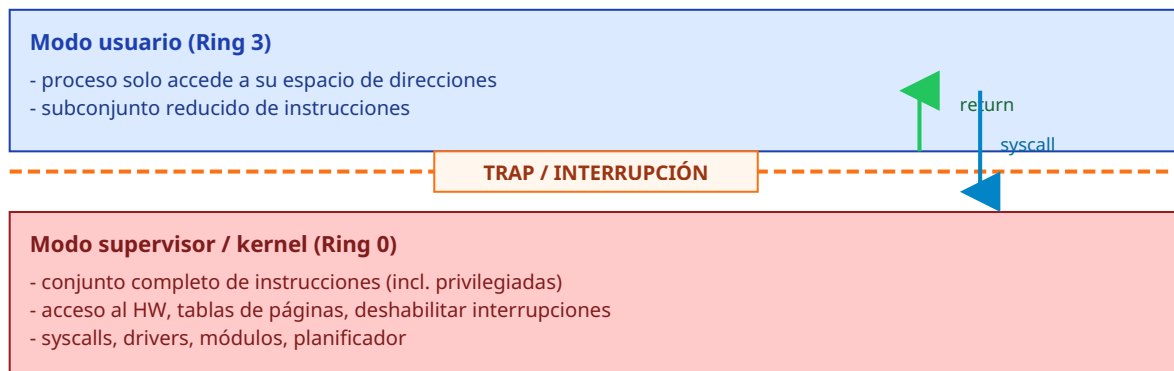


Fig. 1.2 — El bit de modo divide el mundo en dos. La barrera se cruza **solo** vía trap/interrupción.

Tipos de kernel

Hay tres arquitecturas clásicas:

- **Monolítico** (Unix, FreeBSD): toda la funcionalidad linkeada en una única imagen, todo corre en modo supervisor. Máxima eficiencia (no hay cambios de modo internos entre subsistemas) pero un bug en cualquier parte cae todo.
- **Microkernel** (Minix, QNX): en modo supervisor solo lo mínimo (procesos, memoria, IPC básica). Los drivers, filesystem, red, etc. corren en **espacio de usuario** como servicios. Da aislamiento y robustez a costa de rendimiento (más cambios de modo).
- **Monolítico híbrido** (GNU/Linux, Windows NT, macOS/XNU): es monolítico pero admite **cargar y descargar módulos en tiempo de ejecución**, sin recompilar ni reiniciar. Es el modelo de Linux.

Que Linux sea "**monolítico híbrido**" quiere decir que su base es una única imagen monolítica y sus módulos se cargan en caliente. Cuidado con la trampa: aunque el módulo se cargue dinámicamente, una vez cargado **corre en modo kernel** exactamente igual que el resto — un bug en un módulo puede provocar un Kernel Panic.

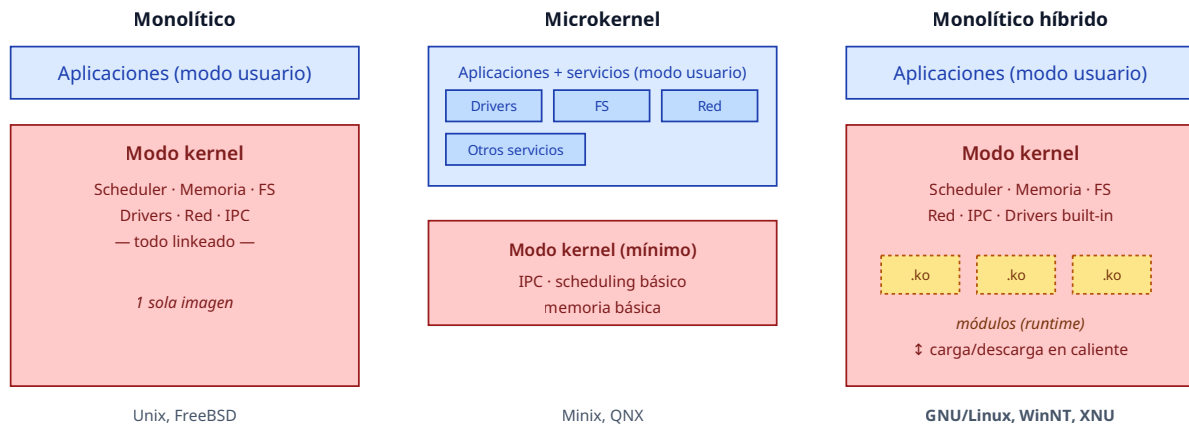


Fig. 1.1 — Los tres tipos de kernel. Linux es monolítico híbrido: base linkeada + módulos `.ko` cargables en caliente, pero todo ejecuta en modo supervisor.

Portabilidad

Linux es **altamente portable**: una misma estructura de código fuente soporta muchas arquitecturas de CPU (x86, ARM, RISC-V, PowerPC, etc.). Esto es posible porque está escrito **mayormente en C** —fácilmente adaptable/recompilable a cualquier arquitectura— reservando **Assembler solo para lo específico de bajo nivel** de cada plataforma. Desde la versión 6.1 también admite Rust, sobre todo para módulos.

Detalle importante: la portabilidad **no significa** que el **mismo binario** corra en cualquier arquitectura. Significa que el **código fuente** se adapta y se recompila. El binario de x86 no arranca en ARM.

Por qué recompilar el kernel

Los motivos típicos son: **soportar nuevos dispositivos** (drivers de hardware nuevo), **agregar funcionalidad** (soporte para nuevos filesystems, protocolos), **optimizar** el rendimiento según el HW donde corre, **adaptar/limpiar** quitando soporte no usado (kernel más liviano) y **corregir bugs** de seguridad o programación. Cosas como “agregar nuevos usuarios” no tienen nada que ver con recompilar el kernel.

Los siete pasos de la compilación (en orden)

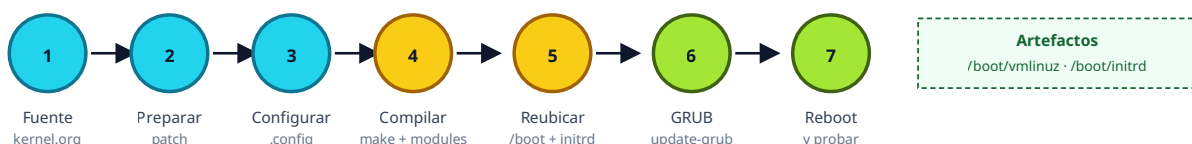


Fig. 1.3 — Flujo canónico de compilación e instalación de un kernel Linux.

1. **Obtener el código fuente** (típicamente desde `kernel.org`).

2. **Preparar el árbol de archivos** (descomprimir, aplicar parches).
3. **Configurar el kernel** (generar el `.config`).
4. **Construir el kernel e instalar los módulos** (`make && make modules_install`).
5. **Reubicar el kernel y crear el initrd/initramfs** (`make install` lo automatiza en muchas distros).
6. **Configurar el gestor de arranque** (GRUB / LILO) para que reconozca el nuevo kernel.
7. **Reiniciar y probar** el nuevo kernel.

Al hacer `make modules_install && make install`, los **módulos** se copian a `/lib/modules/<versión>/` y la **imagen del kernel** (`vmlinuz` / `bzImage`), el `System.map`, el `.config` y el `initramfs` se copian a `/boot/`. Después hay que actualizar GRUB.

Herramientas que se usan: `gcc` (compilador), `make` (ejecuta las directivas de los Makefiles), `binutils` (assembler/linker), `libc`, `ncurses` (solo si usás `menuconfig`) y `initrd-tools`. `make` compara la fecha de modificación de cada target con sus dependencias y solo recompila lo desactualizado; se le puede pasar `-j$(nproc)` para paralelizar.

Configuración (`make config`): la variante `menuconfig` es la más usada (menús en terminal, no requiere entorno gráfico, ideal por SSH); `config` es texto secuencial y `xconfig` es GUI. En `menuconfig`, la barra espaciadora cicla entre `<*>` (built-in, va dentro de la imagen), `<M>` (módulo cargable) y `< >` (deshabilitado).

Parches: un parche es un archivo de `diff` con los cambios respecto a una versión base. Se aplica con el comando `patch`, típicamente por pipe: `xzcat ../patch-6.13.7.xz | patch -p1` (el `-p1` ignora el primer nivel `a/ / b/`). Pueden ser incrementales (uno sobre otro) o no incrementales (sobre el mainline). `patch --dry-run` simula sin modificar.

Por qué hay que reconfigurar el gestor de arranque

Una vez compilado e instalado el nuevo kernel, GRUB **no lo detecta automáticamente**. Hay que ejecutar `update-grub` (o equivalente) para **actualizar el menú de arranque** incorporando el nuevo kernel en `grub.cfg`. Es fundamental entender que esto **no reinstala el binario del gestor**: solo actualiza el archivo de configuración con las rutas de la nueva imagen y su `initramfs`. Por eso podés tener varios kernels compilados en la misma máquina y elegir cuál arrancar desde el menú de GRUB.

initramfs / initrd

Es un **sistema de archivos temporal en RAM** que se monta durante el arranque, **antes** de montar el filesystem raíz real. Contiene los **módulos, drivers y programas mínimos** que el kernel necesita para poder llegar al root definitivo (drivers del controlador de disco, del filesystem del root, LVM/RAID/cifrado si aplica, un shell mínimo tipo `busybox`, `udev` y el `init` temporal).

Una vez que el kernel monta el root real, el initramfs se desmonta y le cede el control al init definitivo.

Puede no ser necesario si todos esos drivers y filesystems críticos están compilados **built-in** (`<*>`) dentro del kernel: entonces el kernel monta el root directamente sin ayuda. En un desktop común esto no es habitual porque el kernel genérico de la distro trae casi todo como módulo para ser universal.

Built-in vs módulo

Cuando configurarás una funcionalidad en el `.config`, tenés tres opciones:

- **Built-in** (`<*>`): compilada dentro de la imagen del kernel. Acceso directo, sin carga en runtime, pero ocupa memoria aunque no la uses y para modificarla tenés que recompilar y reiniciar.
- **Módulo** (`<M>`): archivo `.ko` cargable en caliente. Solo consume memoria cuando se necesita, se puede actualizar sin reiniciar (`insmod` / `rmmod`) y facilita el desarrollo. Contra: corre en modo kernel igual que el resto — un bug puede colgar el sistema.
- **Deshabilitada** (`<>`): no está.

▲ TRAMPAS TÍPICAS DEL PARCIAL

- "Linux es monolítico híbrido **porque** permite cargar/descargar funcionalidades vía módulos" → `V`.
- "Linux es portable porque su código puede modificarse fácilmente para distintas arquitecturas" → `V`. NO es porque el mismo binario corra en cualquier arquitectura.
- "Reconfigurar el gestor de arranque = reinstalar GRUB" → `F`. Solo actualiza el menú.
- "El initramfs siempre es imprescindible" → `F`: si todo lo necesario para montar el root está built-in, no hace falta.
- "El kernel de Linux es un SO en sentido estricto porque contiene todo lo necesario para gestionar HW y procesos" → `F`: es el núcleo; el SO completo agrega utilidades, bibliotecas, shell, etc.
- "En un microkernel los drivers corren en modo kernel para mejorar rendimiento" → `F`: corren en espacio de usuario. Correr todo en modo kernel es lo del monolítico.
- "Se puede tener más de un kernel compilado en la misma máquina" → `V`: GRUB los lista, permite volver atrás si el nuevo no arranca.

2. Módulos, drivers, /dev, /proc y System Calls

Módulos (LKM — Loadable Kernel Modules)

Un **módulo del kernel** es un fragmento de código (archivo `.ko`, *kernel object*) que puede **cargarse y descargarse en tiempo de ejecución sin recompilar ni reiniciar** el kernel. Se instala en `/lib/modules/<versión>/`, comparte el espacio de direcciones del kernel y **se ejecuta en modo supervisor** (privilegiado). Por eso, un bug en un módulo puede llevarse todo el sistema por delante (Kernel Panic en Linux / BSOD en Windows), no solo el proceso que lo invocó.

Todo módulo define una **función de inicialización** (`module_init`, se corre al cargar; devuelve 0 si todo salió bien) y una **función de cleanup** (`module_exit`, se corre al descargar). Los módulos pueden registrar entradas en `/proc` (no es exclusivo de los built-in) y también podés compilar la misma funcionalidad **built-in** dentro de la imagen — en ese caso ya no podés descargarla en caliente.

Los **comandos** clave son:

Comando	Función
<code>insmod <archivo.ko></code>	Carga UN módulo por ruta; no resuelve dependencias, falla si faltan símbolos.
<code>modprobe <nombre></code>	Carga por nombre desde <code>/lib/modules</code> , resuelve automáticamente dependencias usando <code>modules.dep</code> .
<code>rmmod <nombre></code>	Descarga el módulo (solo si su contador de uso está en 0).
<code>lsmod</code>	Lista módulos cargados (es una vista “linda” de <code>/proc/modules</code>).
<code>modinfo <nombre></code>	Muestra metadatos del módulo (autor, licencia, descripción, params).

El archivo `/lib/modules/<versión>/modules.dep` es el mapa de dependencias entre módulos. Lo genera automáticamente `depmod -a` y lo consulta `modprobe` para cargar en el orden correcto todo lo que un módulo necesita. `insmod`, en cambio, no consulta este archivo y falla si le faltan dependencias.

Drivers

Un **driver** es código que actúa de **intermediario entre el kernel y un dispositivo de hardware**. Traduce operaciones genéricas (`open`, `read`, `write`, `close`, `ioctl`) a las instrucciones concretas del hardware particular. En Linux la **forma correcta** de implementar un driver es

como un módulo (no como servicio, no como syscall, no como kthread) — aunque también podés compilarlo built-in. **Todo driver puede ser un módulo, pero no todo módulo es un driver**: los módulos son un mecanismo genérico de extensión, y los drivers son un uso particular de ese mecanismo.

Los drivers representan aproximadamente el **68.8% del código** del kernel Linux y tienen una tasa de errores unas **7× mayor** que el resto del kernel, lo cual —sumado a que corren en modo privilegiado— los convierte en la principal fuente de inestabilidad.

Muy importante: los drivers **no acceden a las funcionalidades del kernel a través de syscalls**. Corren en espacio de kernel, así que invocan **directamente la API interna del kernel** (`kmalloc`, `printk`, `register_chrdev`, `copy_to_user`, `copy_from_user`, etc.). Las syscalls son el mecanismo del espacio de usuario para pedir servicios al kernel.

La función `register_chrdev(major, "nombre", &fops)` registra un dispositivo de carácter en el kernel, asociando un **major number** con una **tabla de operaciones** (`struct file_operations`). Solo puede invocarse desde **espacio de kernel** (código del driver o módulo). La `struct file_operations` es una tabla que indica al kernel qué función del driver ejecutar ante cada operación: `read` → `memory_read`, `write` → `memory_write`, etc. Su inverso es `unregister_chrdev`.

Major y minor number

Cada device file tiene dos números asociados:

- **Major number**: identifica el **driver** asociado a un **tipo** de dispositivo. Le dice al kernel a qué driver derivar la operación.
- **Minor number**: identifica el **dispositivo concreto o instancia** dentro de ese driver. Por ejemplo, `/dev/sda1` y `/dev/sda2` comparten major (mismo driver de disco SCSI) pero difieren en minor (particiones distintas).

/dev — device files

`/dev` contiene los **device files**: archivos que representan dispositivos y son el **punto de acceso desde espacio de usuario** al driver. Encajan con la filosofía UNIX de “todo es un archivo”: los abrís, lees, escribís y cerrás como cualquier archivo, y el kernel se encarga de redirigir esas operaciones al driver adecuado usando el major number.

Los device files **no contienen datos**; son “punteros” al driver. Se crean con `mknod [-m <mode>] <ruta> [b|c] <major> <minor>`. En un `ls -l` aparecen con un prefijo especial:

Tipo	Marca	Acceso	Ejemplos
Carácter	<code>c</code> (<code>crw-...</code>)	Byte a byte, secuencial	teclado, mouse, tty, <code>/dev/null</code> , <code>/dev/random</code>
Bloque	<code>b</code> (<code>brw-...</code>)	Bloques de tamaño fijo, acceso aleatorio	discos: <code>/dev/sda</code> , <code>/dev/nvme0n1</code>

Los **dispositivos de red** NO se representan como archivos en `/dev`. Se acceden a través de la API de sockets y se identifican por nombre de interfaz (`eth0`, `wlan0`). En `/dev` también podés encontrar enlaces simbólicos (`/dev/stdin` → `/proc/self/fd/0`) y sockets/pipes especiales.

/proc — pseudo-filesystem

`/proc` es un **filesystem virtual** que expone las **estructuras internas del kernel** como archivos. **No ocupa espacio en disco**: la mayoría de sus archivos tienen tamaño 0 y su contenido se **genera al vuelo** al leerlo (se ejecuta una función del kernel que devuelve la información). Contiene datos de cada proceso (`/proc/<pid>/`), información del sistema y también configuración escribible.

Un módulo cargable (no solo los built-in) **puede registrar entradas en** `/proc`. Es un mito frecuente en el parcial pensar que solo los built-in pueden.

System Calls

Una **system call** es la **API que expone el kernel al espacio de usuario** para pedirle servicios que requieren modo privilegiado (acceder a archivos, dispositivos, red, crear procesos, etc.). Es el puente controlado usuario → kernel. El mecanismo es una **interrupción de software (TRAP)**: `int 0x80` en x86 de 32 bits o la instrucción `syscall` en x86-64. Todas las syscalls usan la misma interrupción; se distinguen entre sí por el **número** cargado en el registro `EAX` / `RAX`, y admiten hasta 6 parámetros en registros. Al recibir el trap, el **dispatcher** busca el número en la **tabla de syscalls** (`syscall_64.tbl` en x86-64) y ejecuta el handler correspondiente.

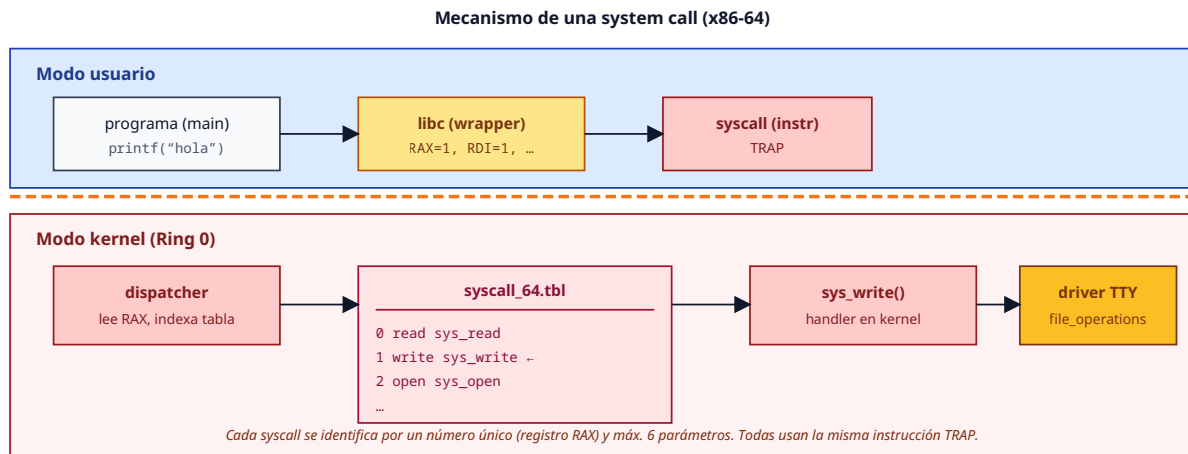


Fig. 2.1 — Ciclo de una syscall: programa → wrapper de libc → instrucción `syscall` (trap) → dispatcher del kernel → tabla → handler → (opcionalmente) driver.

Puntos que suelen confundir:

- Las syscalls **se definen en el código del kernel**, no en `libc` ni en módulos.
- Se **adjuntan estáticamente en tiempo de compilación** del kernel.
- Para **agregar una nueva** hay que (a) declararla en la tabla (`syscall_32.tbl` / `syscall_64.tbl`) con un **número único**, (b) implementar el handler en espacio de kernel y (c) recompilar. No alcanza con reiniciar.
- `libc` **es una biblioteca de usuario** que provee wrappers: acomoda los argumentos, carga el número de syscall y dispara el trap. NO implementa la syscall en sí. La función `syscall()` de la propia libc permite invocar una syscall por número **sin** wrapper específico.
- **POSIX** es un estándar de interfaz que busca la portabilidad de aplicaciones entre Unix; libc respeta POSIX. No toda función de libc es una syscall (muchas son puramente de biblioteca), y técnicamente se puede invocar una syscall sin libc (assembler directo).
- Para monitorear qué syscalls invoca un proceso en runtime se usa `strace`; para mapear nombre ↔ número estáticamente, `ausyscall`.
- Manejo cuidadoso de punteros de usuario: hay que validar con `get_user` / `put_user` / `copy_from_user` / `copy_to_user`. Un puntero mal usado puede provocar Kernel Panic o fugar memoria del kernel.

Todo proceso arranca con **tres file descriptors abiertos**: `0` = stdin (entrada), `1` = stdout (salida), `2` = stderr (errores). Por eso un `printf` termina en un `write(1, ...)`.

▲ TRAMPAS TÍPICAS DEL PARCIAL

- “`libc` es el componente del kernel donde se definen las syscalls” → **F**. `libc` es wrapper de usuario.
- “Las syscalls se implementan como módulos del kernel administrables con `insmod` / `rmmmod`” → **F**. Son parte del kernel base, estáticas.
- “Las syscalls se ejecutan en modo privilegiado y se identifican por un número” → **V**.
- “Cada driver implementa una system call” → **F**. Implementa `file_operations`, no una syscall.
- “Los drivers acceden a las funcionalidades del kernel a través de syscalls” → **F**. Acceso directo por API interna.
- “`register_chrdev` puede invocarse desde espacio de usuario” → **F**. Solo espacio de kernel.
- “Los módulos se ejecutan en espacio de usuario” → **F**. Modo supervisor.
- “Un módulo permite implementar syscalls dinámicas” → **F**.
- “Solo los módulos built-in pueden crear entradas en /proc” → **F**. También los cargables.
- Sobre el output `brw-rw---- 1 root root 240, 0 weird`: la `b` inicial indica dispositivo de bloques y **240** es el **major** (identifica al driver, NO son 240 bytes de tamaño). Escribir con `echo "hola" > weird` no incrementa el tamaño porque no es un archivo regular, es un punto de acceso al driver.

3. Threads (hilos)

Proceso vs hilo

Un **hilo** (o *lightweight process*) es la **unidad básica de ejecución** dentro de un proceso. En los SO modernos, el hilo es la unidad de utilización de CPU / planificación, mientras que el **proceso** es la unidad de **propiedad de recursos**: un proceso es una colección de uno o más hilos que comparten el mismo espacio de direcciones y los mismos recursos.

Los hilos de un mismo proceso **comparten**:

- El espacio de direcciones completo: **código (TEXT), datos, heap**.
- Los recursos del proceso: **archivos abiertos, señales, sockets**.
- El **PID** del proceso.
- El único **PCB** (Process Control Block).

Y cada hilo tiene lo suyo:

- **Stack propio.**
- **Registros y Program Counter (PC)** propios.
- **Estado de ejecución** y contexto del procesador.
- Su propio **TCB** (Thread Control Block) con un **TID** único.

Por eso los hilos son “más baratos”: crear/destruir un hilo solo requiere un TCB + stack + registros, y el cambio de contexto entre hilos del mismo proceso no cambia el espacio de direcciones (solo cambia registros y PC). En cambio, un proceso nuevo obliga al SO a crear un espacio de direcciones aislado, un PCB, etc. La contrapartida es que como los hilos comparten memoria, la **protección entre hilos es responsabilidad del programador** (mutex, semáforos, etc.), no del SO.

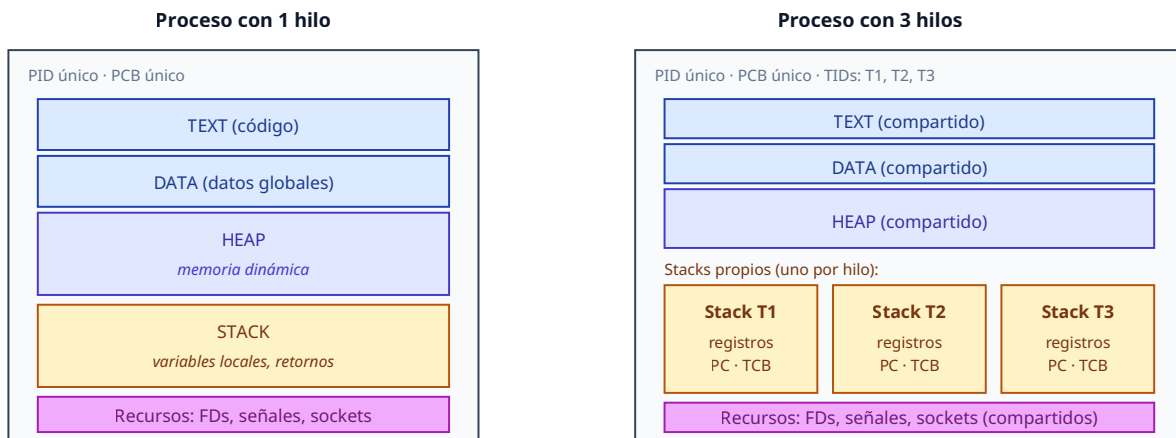


Fig. 3.1 — Qué comparten y qué es propio. TEXT/DATA/HEAP y recursos son del proceso; stack, registros, PC y TCB son de cada hilo.

Concurrencia vs paralelismo

- **Concurrencia:** múltiples tareas **progresan intercaladas** en el tiempo. Puede darse con un único core (multiplexación temporal): da la ilusión de simultaneidad.
- **Paralelismo:** múltiples tareas se ejecutan **realmente al mismo tiempo** en **múltiples cores**.

Todo paralelismo implica concurrencia, pero no toda concurrencia es paralelismo. Un programa concurrente puede correr sin paralelismo en un uniprocador.

ULT vs KLT

Hay dos tipos de hilos según quién los gestiona:

- **ULT (User Level Threads):** la **biblioteca de hilos en espacio de usuario** los crea, gestiona y planifica (por ejemplo GNU Pth, versiones de Pthreads sin soporte de kernel). El **kernel no sabe que existen**: ve un único proceso con un único flujo de ejecución.
- **KLT (Kernel Level Threads):** los **conoce y planifica el kernel**, hilo por hilo. En Linux se crean con la syscall `clone()` / `clone3()` con los flags `CLONE_VM` + `CLONE_THREAD`.

La comparación clave:

Criterio		
¿Quién planifica?	La biblioteca (espacio de usuario)	El kernel
Visibilidad al kernel	Invisible: ve 1 solo proceso	Cada hilo es entidad planificable
Paralelismo en multicore	NO (no puede repartirlos en núcleos)	SÍ (paralelismo real)
Costo de cambio de contexto	Barato (sin pasar a modo kernel)	Caro (cada operación es syscall)
Bloqueo por syscall	Bloquea TODO el proceso	Solo bloquea al hilo llamador
Portabilidad	Alta (funcionan sin soporte del kernel)	Dependen del soporte del SO

El trade-off es claro: los **KLT** ganan en paralelismo real y en robustez ante bloqueos (si un hilo hace una syscall bloqueante, solo se bloquea él, los demás siguen), pero pagan **overhead** por operar en modo kernel. Los **ULT** son rápidos y portables (corren incluso sin soporte del SO) pero **no aprovechan múltiples núcleos** y un bloqueo de un hilo bloquea a todos.

Regla práctica de rendimiento: **KLT / procesos** son mejores para tareas **CPU-bound** (necesitan paralelismo real). **ULT** brillan en **IO-bound**: las esperas se solapan haciendo yields dentro de la biblioteca, sin overhead de kernel.

Modelos de mapeo ULT ↔ KLT

Los ULT no pueden ejecutarse directamente: necesitan mapearse sobre un KLT para obtener CPU. Hay tres modelos:

- **N:1 (Muchos a Uno):** varios ULT sobre un solo KLT. Sin paralelismo real; si un hilo se bloquea, todos se bloquean. Típico de bibliotecas de threads sin soporte del kernel.

- **1:1 (Uno a Uno):** cada ULT tiene su KLT propio. Es el modelo de **Linux, Windows NT y Solaris moderno**. Paralelismo máximo pero costo alto (1 KLT por hilo).
- **M:N (Muchos a Muchos):** M ULT sobre N KLT, con N ajustable. Balance entre paralelismo y costo (Solaris con LWP, TRIX).

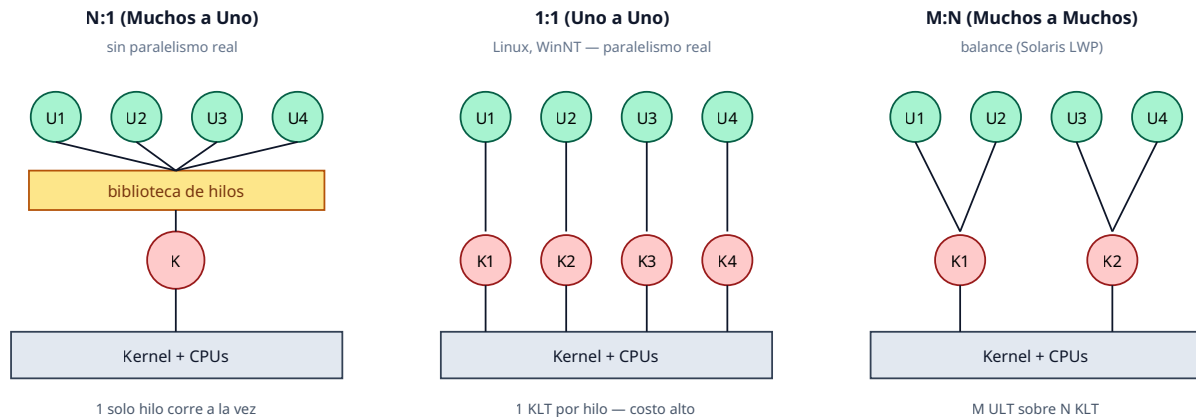


Fig. 3.2 — Modelos de mapeo ULT ↔ KLT. Solo 1:1 y M:N logran paralelismo real en varios núcleos.

fork() y exec()

- **fork()** crea un **proceso hijo**, copia (casi) idéntica del padre. Usa **Copy-On-Write**: padre e hijo comparten las páginas en solo lectura y se copian recién cuando alguna de las partes escribe. Es **fork()** el que asigna el nuevo PID al hijo.
- **exec()** reemplaza la imagen del proceso por otro programa. NO crea proceso ni cambia el PID.
- El patrón clásico de la shell para lanzar un comando externo es **fork() + exec()**: el padre crea un hijo, y el hijo reemplaza su código por el del programa a ejecutar.

fork() sin exec() — cae mucho en parcial. El hijo es una copia (casi) idéntica del padre:

- Ejecuta el **mismo programa/código, mismos datos** que el padre (no se reemplaza).
- **Hereda los descriptores de archivo abiertos**, señales, cwd, etc.
- Tiene **PID propio** (distinto del padre) y su **PPID** apunta al padre.
- Se diferencian por el **valor de retorno de fork()**: **0** en el hijo, PID del hijo en el padre. Cada uno usa esa diferencia para tomar un camino distinto en el código.

En Linux, tanto **fork()** como **pthread_create()** terminan invocando la syscall **clone()**. Lo que cambia son los flags: **fork()** da espacio aislado (COW); **pthread_create()** pasa **CLONE_VM** + **CLONE_THREAD** para compartir espacio de direcciones y recursos con el padre. Por eso **pthread_create** sí aparece en **strace** como un **clone3**, pero **crear ULT no lo hace**: la biblioteca los gestiona internamente sin llamar al kernel.

Detalle: GIL en Python

En CPython el **Global Interpreter Lock (GIL)** permite que solo un hilo ejecute bytecode a la vez, incluso si hay varios cores disponibles. Por eso los hilos no logran paralelismo real en tareas CPU-bound; la solución típica es usar **procesos** (`multiprocessing`), porque cada proceso tiene su propio intérprete y su propio GIL. En tareas IO-bound el GIL se libera durante la espera, así que los hilos sí ayudan.

▲ TRAMPAS TÍPICAS DEL PARCIAL

- “¿Quién planifica los ULT?” → **La biblioteca en espacio de usuario**. “¿Y los KLT?” → **El kernel**.
- “Los ULT en modelo M:1 pueden aprovechar directamente múltiples procesadores físicos” → **F**. Sin mecanismo adicional (KLT), no.
- “Si un proceso crea N ULT, con `strace` se ven N `clone3`” → **F**. Los ULT no invocan al kernel; `clone3` aparece con procesos y con KLT (`pthread_create`), no con ULT.
- “`fork()` sin `exec()`” → hijo es copia del padre, mismo programa, distinto PID, hereda descriptores.
- “¿Quién asigna el PID nuevo?” → `fork()`, no `exec()` (`exec()` conserva el PID).
- Hilo principal: `gettid() = getpid()`. En ULT (GNU Pth) todos comparten el mismo `gettid()` porque el kernel ve una sola entidad; solo `pth_self()` los distingue.
- “Un hilo es la unidad básica de utilización de CPU en los SO modernos” → **V**.

4. Virtualización

Qué es virtualizar y para qué

Virtualizar es una técnica de **abstracción de recursos**: una capa de software desacopla el hardware físico del software que corre encima, oculta detalles y permite que **una única máquina haga el trabajo de varias**, compartiendo el hardware entre múltiples entornos aislados.

Los motivos para virtualizar son variados: **consolidación** (servidores subutilizados que pasan de ~10-18% de uso a ~70%), **aislamiento** (probar apps inseguras o correr software legacy sin contaminar el sistema principal), **aprovechamiento del HW** (varios SO distintos en simultáneo, ahorro energético) y **flexibilidad** (las VMs se encapsulan en archivos, así que backup, copia y migración entre servidores son triviales).

Se distinguen dos actores: el **host** (SO anfitrión + capa de virtualización) y el **guest** (lo que se está simulando: un SO completo).

Hypervisor / VMM

El **hypervisor** o **VMM (Virtual Machine Monitor)** es el programa que implementa las máquinas virtuales, gestiona el reparto de recursos y planifica la ejecución de los guests. Corre en modo supervisor y los guests corren en modo usuario. Cuando el guest intenta ejecutar una instrucción privilegiada, se genera un **trap al VMM**, que la interpreta o emula.

Hay dos tipos:

Dónde corre	Directo sobre el hardware	Como aplicación sobre un SO anfitrión
¿SO anfitrión?	No (el hypervisor es “el SO base”)	Sí
Rendimiento	Mayor	Menor (capa extra)
Uso típico	Servidores, data center	Escritorio, pruebas
Ejemplos	ESXi, Xen, Hyper-V, KVM	VirtualBox, VMware Workstation

La diferencia clave es que el **tipo 1 no necesita SO anfitrión** (corre sobre el bare-metal directamente); el **tipo 2 sí requiere un host** que gestiona los drivers reales del hardware.

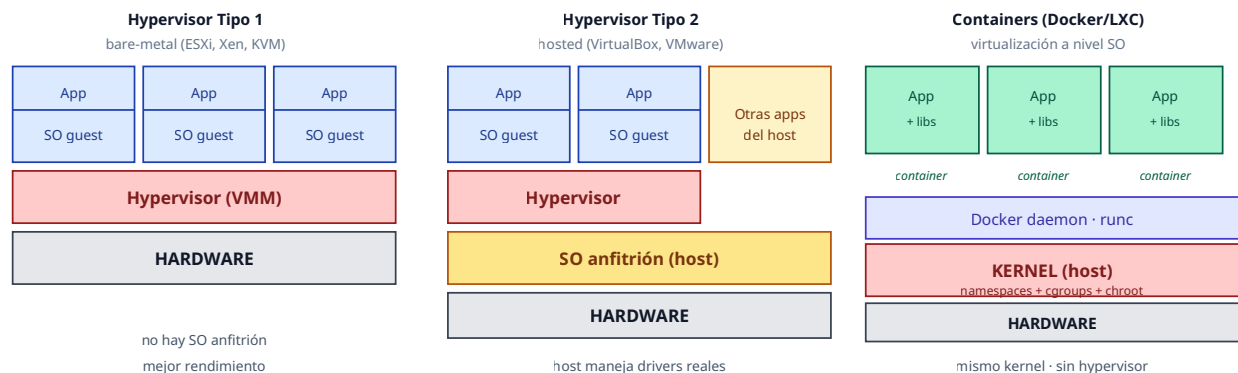


Fig. 4.1 — Tipo 1 vs Tipo 2 vs Containers. Los containers comparten el kernel del host y no usan hypervisor.

Cuidado: el SO guest **nunca tiene acceso directo y completo al hardware real**. El VMM siempre intermedia: crea hardware virtual, controla los recursos e intercepta/emula las instrucciones privilegiadas. El guest ve hardware virtual, no el real.

Técnicas de virtualización

- **Full virtualization con Binary Translation:** el hypervisor **escanea y reescribe en tiempo de ejecución** las instrucciones sensibles del guest (aquellas que en x86 no generaban trap correctamente), reemplazándolas por llamadas seguras al VMM. El **SO guest no se modifica** — no sabe que está virtualizado. Costo alto porque hay que emular todo el HW.
- **Paravirtualización:** el **kernel del SO guest SE MODIFICA** para reemplazar las instrucciones privilegiadas por **hypercalls** (llamadas a la API del hypervisor). **Mejor rendimiento**, pero solo sirve para SO cuyo código puede modificarse. **Windows cerrado NO se puede paravirtualizar.**
- **Virtualización asistida por hardware:** las extensiones **Intel VT-x / AMD-V** en la CPU dan soporte directo a la virtualización (require flag habilitado en BIOS para VMs). Reduce el overhead del binary translation.
- **Emulación:** simula por software un hardware o arquitectura completo, y **puede correr una arquitectura distinta** de la del host (por ejemplo ARM sobre x86). Es la más lenta (ej. QEMU sin aceleración).

Resumen comparativo:

Técnica	¿Modifica el guest?	Rendimiento	Nota
Binary translation	No	Medio	Reescribe instr. privilegiadas en runtime
Paravirtualización	Sí (kernel modificado)	Alto	Usa hypercalls; no sirve para SO cerrados
Emulación	No	Bajo	Puede simular otra arquitectura

Contenedores (virtualización a nivel SO)

LXC / Docker son "virtualización a nivel SO":

- **NO usan hypervisor.**
- **Usan el mismo kernel que el host** (no instalan su propio kernel) → mucho más livianos que una VM.
- El aislamiento del espacio de usuario lo dan **namespaces** (vista aislada de recursos) y **cgroups** (límites de consumo).
- **No pueden correr un SO con kernel distinto** al del host (no virtualizan Windows sobre Linux).
- A diferencia de las VMs, los **procesos del container aparecen como procesos en el host** (con distinto PID gracias al PID namespace).

▲ TRAMPAS TÍPICAS DEL PARCIAL

- “En cuál técnica se debe modificar el kernel del guest” → **Paravirtualización**.
- “En paravirtualización el SO guest NO se modifica” → **F**.
- “La paravirtualización permite virtualizar Windows sobre Linux” → **F**.
- “Los containers usan el mismo kernel que el host” → **V**. “Instalan su propio kernel” → **F**.
- “Los containers necesitan habilitar el flag de virtualización en BIOS” → **F**.
- “Docker/containers necesitan un hypervisor tipo 2” → **F**.
- “El SO guest tiene acceso directo y completo al HW real” → **F**: siempre intermedia el VMM.
- “En hypervisor tipo 2 el SO host se ejecuta directamente sobre el hardware y gestiona los drivers reales” → **V**.
- “En VMs los PIDs del guest aparecen como procesos en el anfitrión” → **F** (esto sí pasa con containers).

5. chroot, cgroups y namespaces

Estos son los **tres mecanismos del kernel de Linux** que, combinados, hacen posibles los contenedores. Individualmente:

- **chroot** aísla el **filesystem** (raíz aparente).
- **namespaces** aíslan la **vista** que un proceso tiene de recursos globales.
- **cgroups** **limitan y controlan** el consumo de recursos.

chroot (Service Isolation)

Data de UNIX v7 (1979). **Cambia el directorio raíz (/) aparente** de un proceso y de todos sus hijos. Aislamiento básico del filesystem: el proceso no puede ver ni acceder a archivos o comandos **fuera** de ese nuevo directorio raíz. Al entorno resultante se lo llama **“jail chroot”**. Docker lo usa para fijar el **union-FS** como raíz del contenedor.

Sintaxis: `chroot /new-root-dir comando`

Por sí solo, **chroot** **no aísla procesos, red, PIDs, ni usuarios**: para eso se necesitan namespaces.

cgroups (Control Groups)

Es una característica del kernel que organiza procesos en **grupos jerárquicos** con el fin de **limitar, priorizar, contabilizar y controlar** su uso de recursos (CPU, memoria, E/S, red, etc.). Son **cuatro funcionalidades**, no solo limitación:

- **Limitación (resource limiting):** un grupo no puede exceder el uso de un recurso.
- **Priorización:** un grupo obtiene prioridad sobre otros al competir por un recurso.
- **Accounting:** medición/estadísticas del uso (útil para billing).
- **Control:** freezar (congelar) y reiniciar un grupo de procesos.

Conceptos base:

- **Controlador o subsistema:** componente del kernel específico por recurso (`cpu` , `memory` , `blkio` / `io` , `cpuset` , `pids` , `net_cls` , `freezer` ...).
- **Jerarquía:** árbol de cgroups, representado como directorios en el pseudo-filesystem cgroup montado (`/sys/fs/cgroup`).
- Un proceso **NO necesita ser modificado** para pertenecer a un cgroup; se lo agrega escribiendo su PID en `cgroup.procs` . Los procesos desconocen que hay límites.
- Un proceso creado por `fork` **queda en el MISMO cgroup del padre** (herencia).
- Los límites de un cgroup hijo **no pueden superar** los del padre.

Ejercicio clásico: limitar la CPU de un proceso al 80%. Se resuelve con cgroups (controlador `cpu`):

1. Crear un cgroup dentro del controlador `cpu` .
2. Escribir la **cuota** (`cpu.cfs_quota_us` / `cpu.cfs_period_us` , o `cpu.max` en v2) correspondiente al 80%.
3. Agregar el PID del proceso a `cgroup.procs` .

cgroups v1 vs v2:

Aspecto		
Jerarquías	Múltiples (una por controlador)	Única / unificada
Montar un controlador particular	Sí se puede	No (todo en la jerarquía única)
Pertenencia de un proceso	A un cgroup por cada jerarquía (varias a la vez)	A un solo cgroup en toda la jerarquía
¿Procesos en cgroups internos?	Permitidos	Solo en cgroups sin hijos (hojas) , salvo el root ("no internal process constraint")
Desmontar un controlador	Solo si no tiene cgroups hijos	—

Las dos versiones **pueden coexistir** en el mismo sistema; lo que **no se puede** es montar **el mismo controlador** en ambas simultáneamente.

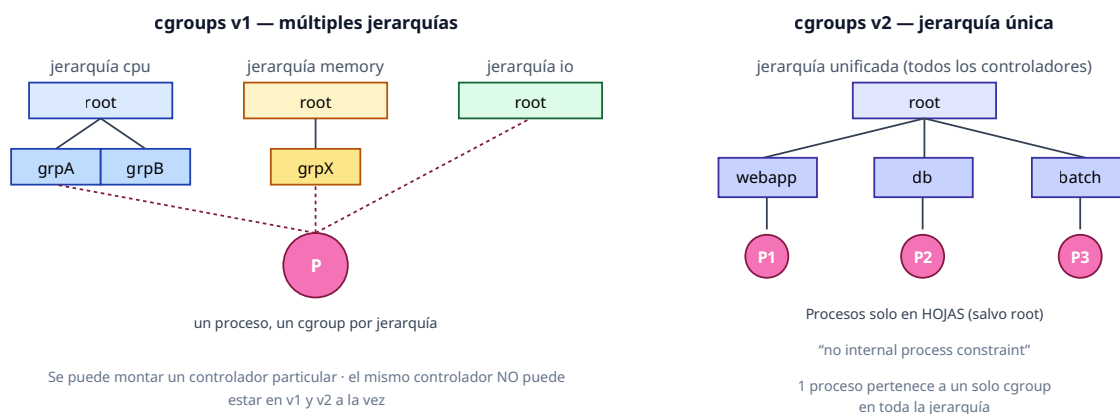


Fig. 5.1 — cgroups v1 (una jerarquía por controlador) vs v2 (jerarquía única, procesos solo en hojas).

Namespaces

Un **namespace** da a un proceso una **vista aislada de un recurso global del sistema**: el proceso "cree" tener su propia instancia. Un proceso está en **un namespace de cada tipo** a la vez, y sus hijos **heredan** los del padre (salvo que se cambien explícitamente).

Namespace	Flag <code>CLONE_*</code>	Qué aísla
PID	<code>CLONE_NEWPID</code>	Árbol de PIDs propio; el primer proceso es PID 1 (init) del namespace
NET	<code>CLONE_NEWNET</code>	Interfaces de red, pilas, IPs, puertos
MNT / Mount	<code>CLONE_NEWNS</code>	Puntos de montaje (tabla de montajes propia)
UTS	<code>CLONE_NEWUTS</code>	Hostname y nombre de dominio
IPC	<code>CLONE_NEWIPC</code>	System V IPC, colas de mensajes POSIX
USER	<code>CLONE_NEWUSER</code>	Mapeo de UIDs/GIDs — root dentro puede no ser root en el host
Cgroup	<code>CLONE_NEWCGROUP</code>	Vista del cgroup root
Time	<code>CLONE_NEWTIME</code>	Offset del reloj por namespace

Las **syscalls** que los manipulan son `clone()` (crea proceso + namespace nuevo), `unshare()` (mueve el proceso actual a un namespace nuevo) y `setns()` (une el proceso a un namespace existente).

Doble PID por PID namespace: un proceso `mi_proceso` corriendo dentro de un contenedor tiene **PID 1** dentro (es el primer proceso de su namespace, su "init") y un **PID real más alto** en el host (por ejemplo 423548). Desde el host se ven **todos** los procesos de todos los contenedores; desde el contenedor **no** se ven los del host.

PID namespace — mismo proceso, dos PIDs

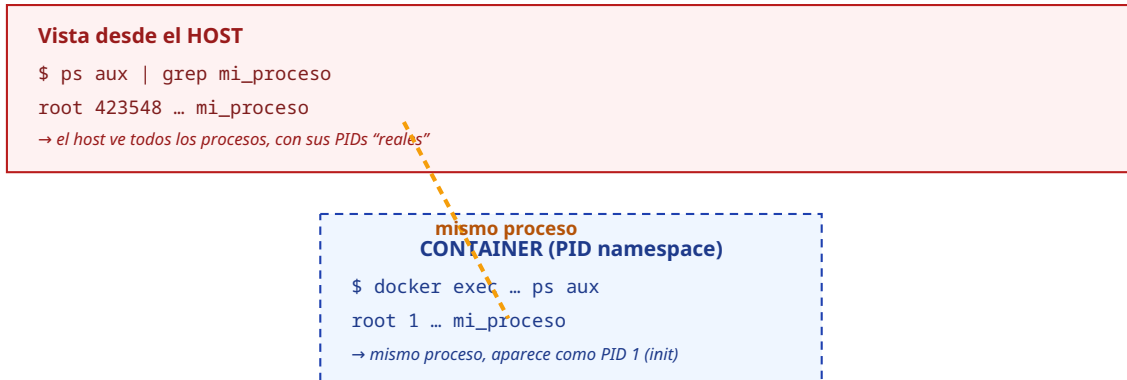


Fig. 5.2 — Un proceso dentro de un container tiene PID 1 en su namespace y un PID real más alto en el host.

▲ TRAMPAS TÍPICAS DEL PARCIAL

- "¿Qué mecanismo del kernel limita el uso de CPU al 80%?" → **cgroups** (controlador **cpu**, cuota).
- "cgroups proveen limitación pero NO priorización" → **F** : proveen ambas.
- "Una vez agregado a un cgroup, un proceso no puede moverse a otro" → **F** .
- "Los procesos deben ser modificados para incorporarse a un cgroup" → **F** .
- "Un proceso creado por **fork** no pertenece al cgroup del padre" → **F** : lo hereda.
- v1: "existe una sola jerarquía en todo el sistema" → **F** (eso es v2).
- v1: "un proceso puede estar en varios cgroups a la vez dentro de una jerarquía" → **F** (uno por jerarquía).
- v2: "los procesos deben agregarse a cgroups sin hijos, salvo el root" → **V** .
- "No es posible tener las dos versiones de cgroups montadas simultáneamente" → **F** : sí se puede; lo que no se puede es el mismo controlador en ambas.
- "Un proceso tiene distinto PID dentro del contenedor y en el host" → **V** (PID namespace).

6. Docker y contenedores

Docker es una **plataforma open-source para empaquetar y ejecutar aplicaciones en contenedores livianos**. Trabaja a **nivel SO**, comparte el kernel del host y **NO usa hypervisor**.

Imagen vs Contenedor

Es la distinción central del tema:

Qué es	Template/molde de solo lectura	Instancia en ejecución de una imagen
Contenido	App + dependencias + librerías + instrucciones	Imagen + capa escribible propia
Ejecución	NO se ejecuta (es estática)	Sí se ejecuta (proceso/s aislado/s)
Relación	De 1 imagen se crean varios contenedores	Cada uno autónomo y aislado
Analogía OOP	Clase	Objeto/instancia

Una imagen puede **basarse en otras** (cadena de capas). Se puede generar una imagen a partir de un contenedor con `docker commit`. La diferencia clave es que la imagen es solo lectura y el contenedor le agrega encima una **capa escribible**.

Qué usa Docker del kernel

Docker corre en **espacio de usuario (Ring 3)**. **No captura syscalls** — eso lo hace el kernel. Pero sí requiere funcionalidades del kernel para armar los contenedores:

- **chroot**: cambia el directorio raíz del contenedor.
- **Namespaces**: vista aislada (PID, Net, Mount, UTS, IPC, User).
- **Cgroups**: limitan y controlan recursos (CPU, memoria, etc.).
- **Union Filesystem**: apila capas de imagen como si fueran un único FS.

El contenedor tiene sus propios filesystem, librerías, red, nombre... **excepto su propio kernel** (lo comparte con el host).

Docker usa **arquitectura cliente-servidor**: `dockerd` es el **servidor / daemon** (crea, ejecuta y monitorea contenedores e imágenes) y la CLI (`docker`) es el **cliente** que se comunica con él vía la API REST.

Union Filesystem y capas

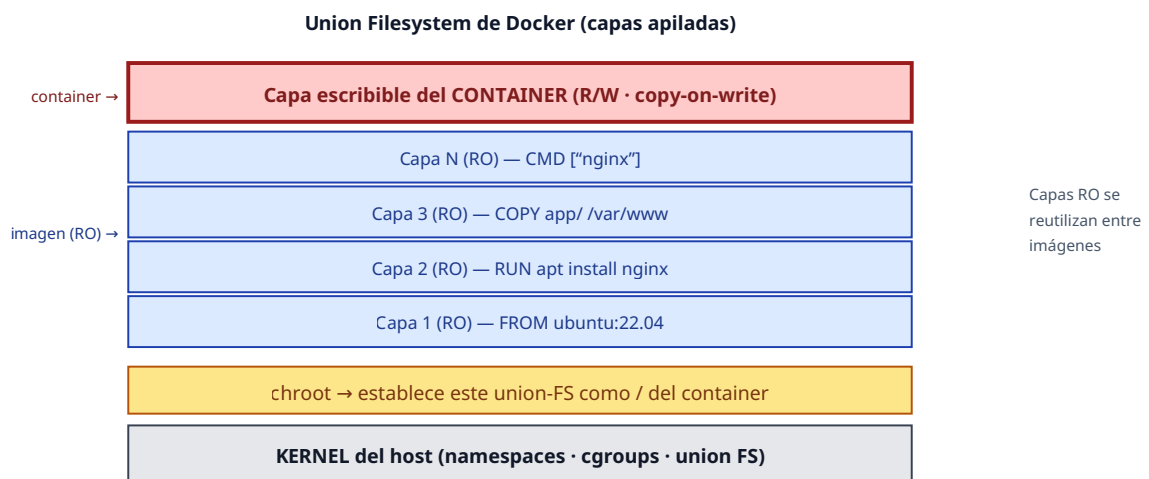
Es un **mecanismo de montaje** (no un FS nuevo): varios directorios montados en **un mismo punto** aparecen como uno solo. Las capas **inferiores son de solo lectura**; la capa **superior es escribible**.

Cada imagen es un conjunto de capas apiladas, donde cada capa guarda **solo las diferencias** respecto de la anterior. **Todas las capas de una imagen son de solo lectura**, y esas capas se **reutilizan entre imágenes** (ahorra espacio y descargas).

Al ejecutar un contenedor (copy-on-write):

1. Docker apila las capas read-only de la imagen.
2. Con `chroot` establece ese union-FS como raíz del contenedor.
3. Agrega encima una **capa escribible propia del contenedor**.
4. Solo esa **última capa** puede modificarse → varios contenedores pueden correr sobre las mismas capas base compartidas.

Al eliminar el contenedor se pierde su capa escribible; las capas inferiores quedan intactas. Por eso todo lo escrito **dentro del contenedor** se pierde si no se guarda fuera; y por eso `RUN rm archivo` en un Dockerfile **no reduce el tamaño de la imagen**: la capa donde estaba el archivo ya quedó fija; el `RUN rm` solo agrega una nueva capa que oculta el archivo, pero el original sigue ocupando en la capa previa.



Nota: `RUN rm foo` genera una capa nueva que oculta el archivo pero NO libera espacio en las capas RO inferiores.

Fig. 6.1 — Union FS: capas RO de la imagen + 1 capa escribible del container. Solo la última capa se modifica.

Containers vs VMs

Comparte el kernel del host	SO guest completo + kernel propio
Liviano, arranque rápido	Pesada, arranque lento
Aislamiento a nivel de proceso	Aislamiento completo (HW virtual)
NO necesita hypervisor	Requiere hypervisor

Dockerfile vs docker-compose

Qué hace	Instrucciones para construir UNA imagen paso a paso	Definición declarativa para levantar y orquestar una app de varios contenedores
Resultado	Una imagen (1 pieza)	Conjunto de contenedores + redes + volúmenes
Lenguaje	Instrucciones (<code>FROM</code> , <code>RUN</code> , <code>COPY</code> , <code>CMD</code> , ...)	YAML (indentación, clave: valor)
Comando	<code>docker build</code>	<code>docker compose up</code> / <code>down</code>

No son excluyentes: compose puede usar `build:` apuntando a un Dockerfile. Cada instrucción del Dockerfile genera **una nueva capa** de la imagen.

Persistencia de datos

Lo escrito en la capa del contenedor se pierde al destruirlo. Para persistir hay que guardarlo en el host y montarlo dentro:

Ubicación	Gestionada por Docker en <code>/var/lib/docker/volumes</code>	Cualquier ruta del host elegida por el usuario
Portabilidad	Más portables, desacoplados del host	Atados a una ruta concreta
Acceso externo	Manejados por Docker	Pueden ser modificados por procesos ajenos a Docker

▲ TRAMPAS TÍPICAS DEL PARCIAL

- “A partir de una imagen solo se puede generar un solo contenedor” → **F** (varios).
- “Docker necesita un hypervisor para ejecutarse” → **F** (trabaja a nivel SO).
- “Un Dockerfile crea un container” → **F** : crea una **imagen**.
- “Cada imagen está compuesta por capas de las cuales solo la última puede modificarse” → **V** (la última es la del container).
- “Docker usa namespaces, cgroups y union filesystems” → **V** .
- “No es posible generar una imagen a partir de un container” → **F** : `docker commit` .
- “Docker no requiere funcionalidades del kernel porque corre en modo usuario” → **F** : requiere chroot, namespaces, cgroups.
- “Todo archivo agregado a un container automáticamente pasa a ser parte de la imagen” → **F** : va a la capa escribible del container, no de la imagen.
- “`RUN rm` reduce el tamaño total de la imagen” → **F** (el archivo queda en la capa anterior).
- “Docker daemon (dockerd) es el servidor que crea/ejecuta/monitorea los contenedores” → **V** .

7. Protección, seguridad y permisos

Protección vs seguridad

- **Protección:** son los **mecanismos internos** del SO que controlan el acceso de procesos y usuarios a los recursos del sistema. Es el “cómo técnico” (los candados).
- **Seguridad:** concepto más general, la **defensa frente a amenazas externas e internas**; mide cuánto se puede confiar en que el sistema y sus datos mantienen su integridad.

La **protección es una parte de la seguridad**.

Hay una distinción clave entre **políticas** y **mecanismos**:

- Las **políticas** definen **qué** se quiere hacer / qué está permitido. Decisión de alto nivel; rara vez incluyen configuraciones.
- Los **mecanismos** son las herramientas, configuraciones e implementaciones concretas que **cómo** hacen cumplir la política.

Primero se define la política, después el mecanismo. Una misma política admite varios mecanismos: eso da flexibilidad (cambiar el mecanismo sin tocar la política).

La seguridad se mide con la **tríada CIA**:

- **Confidencialidad**: evitar la lectura/intercepción no autorizada.
- **Integridad**: evitar la modificación no autorizada.
- **Disponibilidad**: evitar la interrupción del servicio/recursos.

Cuidado con la falacia común: un sistema con muchas features **no** es automáticamente más seguro; al contrario, agrega superficie de ataque (más código, más bugs potenciales). La seguridad se favorece con el **principio de mínimo privilegio (POLA)** y quitando lo que no se usa.

Dominios de protección

Un **dominio de protección** es un conjunto de pares (**objeto, derecho**) que especifica qué operaciones puede realizar un proceso sobre cada objeto. Un proceso “vive” en un dominio y solo puede hacer sobre cada objeto las operaciones autorizadas por ese dominio. En UNIX el dominio de un proceso lo define el par (**UID, GID**).

Permisos UNIX (rwx)

Cada archivo/directorio tiene **tres conjuntos de permisos**, uno por categoría:

- **u** (user/owner): el dueño del archivo.
- **g** (group): el grupo del archivo.
- **o** (others): todos los demás.

Cada conjunto tiene **r** (read = 4), **w** (write = 2) y **x** (execute = 1). En **octal** se suma por categoría. Por ejemplo `rwXr-r--` = $4+2+1 / 4+0+1 / 4+0+0 = 754$.

Comandos:

- `chmod` — cambia permisos (`chmod 754 f` , `chmod u+x f`).
- `chown` — cambia el dueño (`chown usuario f`).
- `chgrp` — cambia el grupo (`chgrp grupo f`).

Permisos especiales: setuid, setgid, sticky

Se representan como un **cuarto dígito octal** que precede a los permisos normales (por ejemplo `chmod 4755`).

Permiso	Octal	Sobre un	Sobre un
setuid (SUID)	4	Corre con los privilegios del DUEÑO del archivo , no del que lo lanza (ej: <code>passwd</code> corre como root)	Generalmente sin efecto
setgid (SGID)	2	Corre con los privilegios del GRUPO del archivo	Los archivos nuevos heredan el grupo del directorio (no el del usuario que los crea)
sticky bit	1	Sin efecto relevante hoy	Solo el DUEÑO del archivo (o el dueño del directorio, o root) puede renombrar/eliminar sus archivos, aunque otros tengan permiso de escritura en el dir. Ejemplo típico: <code>/tmp</code>

Setuid/setgid implementan el **cambio de dominio dinámico** en UNIX: el proceso pasa temporalmente al dominio del dueño/grupo del ejecutable, justo cuando necesita más privilegio. En `ls -l` aparecen como `s` (en lugar de `x`) o `t` (sticky).

UMASK

Es la **máscara** que define los permisos por defecto al crear archivos y directorios. Funciona por **resta**: se quitan sus bits de los permisos base (**666** para archivos, **777** para directorios).

Ejemplo con `umask 022`: archivo nuevo → `666 - 022 = 644`; directorio nuevo → `777 - 022 = 755`.

/etc/passwd vs /etc/shadow

Archivo	Contiene	Permisos
<code>/etc/passwd</code>	Info de usuarios: login, UID, GID, home, shell	Legible por todos (la contraseña NO está acá)
<code>/etc/shadow</code>	Hashes de contraseñas y políticas de expiración	Solo root puede leerlo/modificarlo

ASLR (Address Space Layout Randomization)

Es un mecanismo que **aleatoriza las direcciones** de memoria de un proceso en cada ejecución: **stack, heap, librerías compartidas** y, si el binario es PIE (Position Independent Executable), también el **código**. El objetivo es **dificultar exploits de memoria** (buffer overflow, return-to-libc) que necesitan direcciones fijas conocidas para funcionar.

Linux provee ASLR para procesos de usuario: sí. **Para el kernel también**, mediante **KASLR** (Kernel ASLR), que aleatoriza la dirección base donde se carga el kernel al arrancar. Es un **mecanismo separado** del ASLR de usuario.

Se controla con `/proc/sys/kernel/randomize_va_space`, con tres valores:

- `0` = desactivado.
- `1` = parcial (stack, mmap/librerías, vdso).
- `2` = completo (+ heap/brk). Es el **valor por defecto**.

Se puede activar/desactivar **sin recompilar el kernel** (basta escribir en ese archivo).

Buffer overflow y shellcode (contexto)

- **Buffer overflow:** se escribe más allá del área de memoria reservada. C **no chequea límites**, así que un `strcpy` en un buffer pequeño puede pisar valores adyacentes en la pila, incluyendo la dirección de retorno de la función.
- **Shellcode:** código malicioso preparado para obtener los privilegios del programa atacado (típicamente lanzar una shell).
- ASLR y el uso correcto de permisos especiales (mínimo privilegio) **mitigan** estos ataques.

▲ TRAMPAS TÍPICAS DEL PARCIAL

- “ASLR es solo para procesos de usuario en Linux” → **F** : también para el kernel (KASLR).
- “Con UMASK se indican los permisos por default al crear un archivo/directorio” → **V** .
- “El sticky-bit en un directorio permite que solo el dueño del archivo (o dueño del dir o root) pueda renombrar/eliminar los archivos” → **V** .
- “En Linux `/etc/shadow` solo puede ser modificado por root” → **V** .
- “setuid, setgid y sticky son permisos especiales” → **V** .
- “setuid usa privilegios del que ejecuta el archivo” → **F** : del **dueño** del archivo.
- “La contraseña está en `/etc/passwd` ” → **F** : en `/etc/shadow` (hasheada).
- “Un sistema con muchas features es más seguro porque tiene más herramientas de defensa” → **F** : más features = más superficie de ataque. POLA.
- “Un dominio de protección es un conjunto de pares (objeto, derecho) que especifica operaciones sobre cada objeto” → **V** .

8. File Systems, RAID y LVM

Tema clave y muy preguntado. Nota: si el programa de tu año excluye RAID/LVM, saltealo, pero cubrilo por las dudas — históricamente cae.

Conceptos base de File Systems

Un **file system** organiza cómo se almacenan, nombran y recuperan los datos en un dispositivo. Reglas fundamentales:

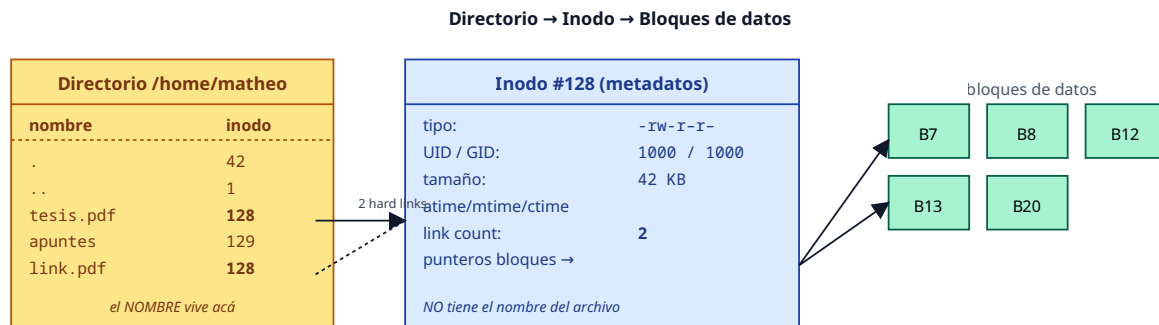
- Una partición **debe tener un FS definido** para poder ser accedida (montada). Sin FS no hay lectura/escritura.
- El FS administra el espacio con **bloques** y describe cada archivo con un **inodo**.
- **El nombre del archivo NO vive en el inodo**: vive en la **entrada de directorio**, que asocia `nombre → número de inodo`.

Inodo: estructura que guarda los **metadatos** del archivo:

- Permisos (rwx), dueño (UID) y grupo (GID).
- Tamaño del archivo.
- Fechas: `atime` (acceso), `mtime` (modificación), `ctime` (cambio de metadatos); en Ext4 también `crtime` (creación).

- Punteros a los bloques de datos (directos, indirectos, doble/triple indirecto).
- **Contador de hard links** (link count).

Se puede montar un FS con `noatime` para no actualizar la fecha de acceso en cada lectura (mejora rendimiento).



Hard link = otra entrada de directorio apuntando al mismo inodo. Symlink = archivo aparte con su propio inodo cuyo contenido es una ruta.

Fig. 8.3 — El directorio asocia nombre → inodo. El inodo tiene los metadatos + punteros a bloques. El nombre **no** vive en el inodo.

Bloque: unidad **mínima de almacenamiento** de datos del FS. Su tamaño se elige al crear el FS (típicamente 1K, 2K, 4K) y **no puede cambiarse dinámicamente después**.

Extent: conjunto de **bloques contiguos** descriptos como un rango (`inicio + longitud`) en vez de bloque por bloque. Ventajas: menos metadatos y mejor rendimiento para archivos grandes. Lo usan Ext4, XFS y BTRFS.

Tipos de File System

EXT2 / EXT3 / EXT4:

- Dividen el FS en **grupos de bloques** (block groups).
- El **superbloque** (info global del FS) está **replicado** en varios grupos de bloques como redundancia ante corrupción.
- **Ext2** no tiene journaling. **Ext3** lo agrega. **Ext4** agrega extents, journaling mejorado y guarda **fecha de creación** (crtime); soporta volúmenes y archivos más grandes.

Journaling: es un **log** donde se anotan las operaciones **antes** de aplicarlas al FS. Permite **recuperar el FS** a un estado consistente tras un corte de energía o caída. **Cuidado:** NO almacena “todas las operaciones sobre un archivo”; registra operaciones de metadatos / transacciones pendientes de aplicar, para recuperación.

XFS:

- Los **inodos se asignan dinámicamente** (no hay una tabla fija reservada al crear el FS).
- Alto rendimiento, usa extents y journaling.

- **NO se puede achicar** un FS XFS: solo se puede **crecer** (extender).

BTRFS:

- Usa **Copy-on-Write (CoW)**: al modificar un dato, no sobrescribe el bloque original; escribe en un bloque nuevo y actualiza los punteros / árbol. Esto da consistencia ante cortes (nunca queda a medias) y **snapshots eficientes y baratos** (comparten bloques, solo se copia lo que cambia).
- Soporta **subvolúmenes**: raíces de árbol de archivos independientes dentro del mismo FS.
- En un subvolumen **no se puede definir un FS distinto a BTRFS** (el subvolumen es siempre BTRFS).
- Para **montar un subvolumen que ocupa más de una partición** basta con indicar **una sola** de esas particiones (BTRFS/LVM resuelven el resto).
- Soporta RAID interno, checksums y compresión.

Links: hard vs simbólico

Característica	HARD LINK	SYMLINK (soft)
Apunta a...	El mismo inodo que el original	Una ruta / nombre
¿Consume inodo nuevo?	NO (comparte el inodo)	SÍ (tiene su propio inodo)
Si se borra el original	Sigue funcionando (los datos viven hasta que link count = 0)	Queda roto
¿Cruza particiones / FS distintos?	NO	sí

Clave del inodo: se elimina recién cuando su contador de links llega a 0. Por eso un hard link mantiene vivos los datos aunque borres el nombre original.

RAID

RAID (Redundant Array of Independent Disks) combina varios discos físicos en una unidad lógica para mejorar velocidad, capacidad y/o redundancia. Conceptos base:

- **Striping**: reparte los datos en varios discos, en paralelo → velocidad.
- **Mirroring**: copia idéntica de los datos en otro disco → redundancia.
- **Paridad**: información calculada que permite **reconstruir** datos ante un fallo.

Nivel	Striping	Paridad	Redundancia	Capacidad útil	Tolera fallo de
RAID 0	Sí	No	No	Suma de todos (100%)	0 discos
RAID 1	No	No	Sí (espejo)	La de 1 disco (50% con 2)	1 disco
RAID 4	Sí	Dedicada (1 disco)	Sí	N-1 discos	1 disco
RAID 5	Sí	Distribuida entre todos	Sí	N-1 discos	1 disco
RAID 6	Sí	Doble distribuida	Sí	N-2 discos	2 discos

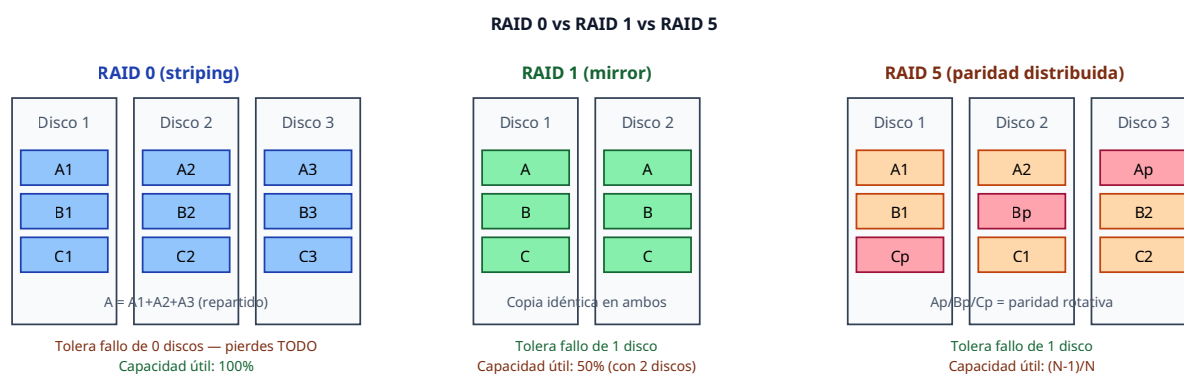


Fig. 8.2 — Comparación visual de los niveles clásicos de RAID. En RAID 5 la paridad rota entre los discos (bloques P_a, P_b, P_c).

Cálculos típicos del parcial:

- RAID 5 con **5 discos de 1 TB** → capacidad útil = **(N-1) = 4 TB**. Tolera **1 disco caído**.
- Discos de distinto tamaño: el RAID se limita al **disco más chico**. Con 1, 2 y 3 GB → el array usa 3×1 GB efectivos; en RAID 5 la capacidad útil sería **2 GB**.

Chunk size: tamaño del bloque de striping (cuánto se escribe en un disco antes de pasar al siguiente). **No depende de la cantidad de discos.**

DDP (Dynamic Disk Pool): distribuye datos, paridad y spare sobre todos los discos de un pool, y la reconstrucción es más rápida. **No** funciona como “11 discos fijos reservando 2”.

LVM (Logical Volume Manager)

Capa de abstracción sobre los discos físicos que da **flexibilidad** frente al particionado clásico. Su gran ventaja: permite **extender un FS a través de diferentes particiones e incluso discos rígidos distintos**.

Jerarquía LVM:

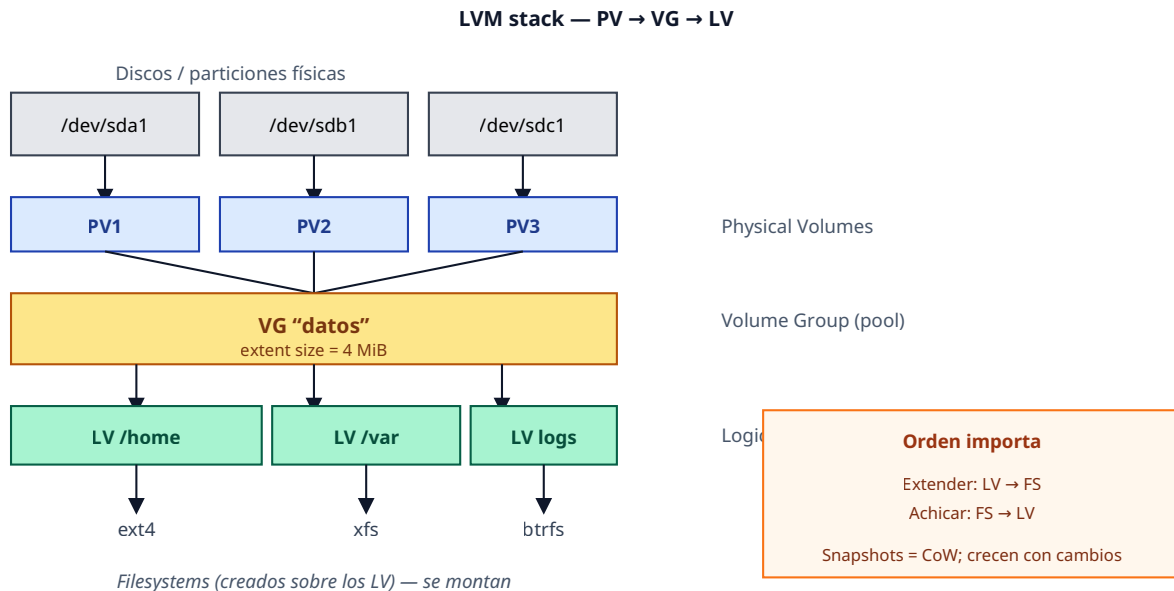


Fig. 8.1 — Stack LVM: los PV inicializan particiones para LVM, el VG los agrupa como pool con un extent size, y los LV se tallan del pool para montar FS encima.

Reglas clave:

- El **tamaño de un LV** es siempre **múltiplo del tamaño del EXTENT** definido en el VG.
- Un LV puede **abarcas varios discos**: no está limitado al espacio de un único disco físico.

Orden de operaciones (mnemotecnía):

- **Extender (agrandar):** primero el **LV**, después el **FS**.
- **Achicar (reducir):** primero el **FS**, después el **LV**.
- Regla: "para crecer, primero el contenedor; para achicar, primero el contenido" — así no se pierden datos.

Snapshots LVM:

- Usan **Copy-on-Write**.
- Al crear el snapshot **no se copian** los datos ni metadatos del LV original: se copian **a medida que el LV original se modifica**, y solo los bloques que cambian.
- El **espacio del snapshot crece** con las modificaciones del original.
- **No se eliminan solos**: hay que borrarlos manualmente (si se llena, se invalidan).

▲ TRAMPAS TÍPICAS DEL PARCIAL

- “RAID 1 provee striping y paridad distribuida” → **F** : RAID 1 es mirroring, sin striping ni paridad.
- “¿En qué nivel de RAID NO existe striping?” → **RAID 1**.
- “RAID 5 con 5 discos de 1 TB tiene capacidad útil de...” → **4 TB** (N-1).
- “¿Cuántos discos pueden fallar en RAID 5 sin pérdida?” → **1**.
- “El chunk size depende de la cantidad de discos” → **F** .
- “El nombre del archivo está en el inodo” → **F** : en el directorio.
- “Cada vez que se crea un hard link se usa un nuevo inodo” → **F** : comparte inodo.
- “Si se elimina un symlink, se elimina el archivo apuntado” → **F** .
- “Si se elimina el archivo original, el symlink queda roto” → **V** .
- “En XFS los inodos se asignan dinámicamente pero no se puede reducir el FS” → **V** .
- “¿En qué FS no se puede disminuir el tamaño?” → **XFS**.
- “El journaling almacena todas las operaciones sobre un archivo” → **F** : es un log de operaciones pendientes de aplicar, para recuperación.
- “En BTRFS un subvolumen puede tener un FS distinto a BTRFS” → **F** .
- “Para montar un subvolumen que ocupa más de una partición basta con indicar una sola” → **V** .
- “El tamaño de un LV siempre es múltiplo del extent del VG” → **V** .
- “Un LV solo se puede extender si hay espacio en el disco físico donde está definido” → **F** : puede abarcar varios.
- “Para achicar un LV: primero el FS y después el LV” → **V** (para extender, al revés).
- “Los snapshots LVM usan CoW y crecen con las modificaciones del LV original” → **V** .
- “Al crear el snapshot se copian metadatos y datos del LV original” → **F** .
- “Una partición debe tener un FS definido para poder ser accedida” → **V** .

9. Multiprocesadores

Por qué multiprocesadores

Históricamente la industria buscó más poder de cómputo subiendo la **velocidad de reloj**. Hoy eso choca con dos límites físicos: ninguna señal viaja más rápido que la luz (~20 cm/ns en cobre/fibra) y la **disipación de calor** y el consumo eléctrico crecen mucho más rápido que el

rendimiento. La solución actual es **cómputo paralelo/distribuido**: varias CPU a velocidad “normal” que en conjunto dan la potencia. Más cores, no más Hz.

Esquemas de arquitectura (mayor → menor acoplamiento)

Esquema	Memoria	Comunicación	Retardo
Multiprocesador (memoria COMPARTIDA)	Un único espacio de direcciones, por BUS	A través de la memoria compartida	2–10 ns
Multicomputadora (memoria DISTRIBUIDA)	Cada CPU tiene su memoria local	Pasaje de mensajes por interconexión de alta velocidad	10–50 μs
Sistema distribuido	Cada nodo es una PC completa	Pasaje de mensajes por red	10–100 ms

Las **multicomputadoras** (clusters) son fáciles de fabricar pero difíciles de programar; son **fuertemente acopladas** (una tarea empieza y termina en la misma CPU). Los **sistemas distribuidos** son PCs completas y heterogéneas (distintos SO/HW) conectadas por red. El salto de retardo de nseg a mseg es un factor de millones.

Memoria compartida: UMA vs NUMA

A nivel HW, en un multiprocesador de memoria compartida cada procesador direcciona toda la memoria. Según la velocidad de acceso:

Tiempo de acceso	Igual para todas las CPU	Depende: local rápido / remoto lento
Espacio de direcciones	Único	Único (visible a todas)
Acceso remoto	—	Con LOAD/STORE , requiere bus compartido
Escalabilidad	Baja (caro)	Alta
Ejemplo típico	SMP por bus	Grandes sistemas multi-nodo

UMA basado en bus: un único bus, y al sumar CPU el **ancho de banda del bus** se vuelve cuello de botella (CPU ociosa esperando el bus). Se palia con **caches** (bloque RO en varias caches; RW en una sola, requiere protocolo de coherencia). Para escalar más allá del bus

(~16-32 CPU) se usan crossbar (barras cruzadas) o redes multietapa, con distintos trade-offs de contención vs cantidad de switches.

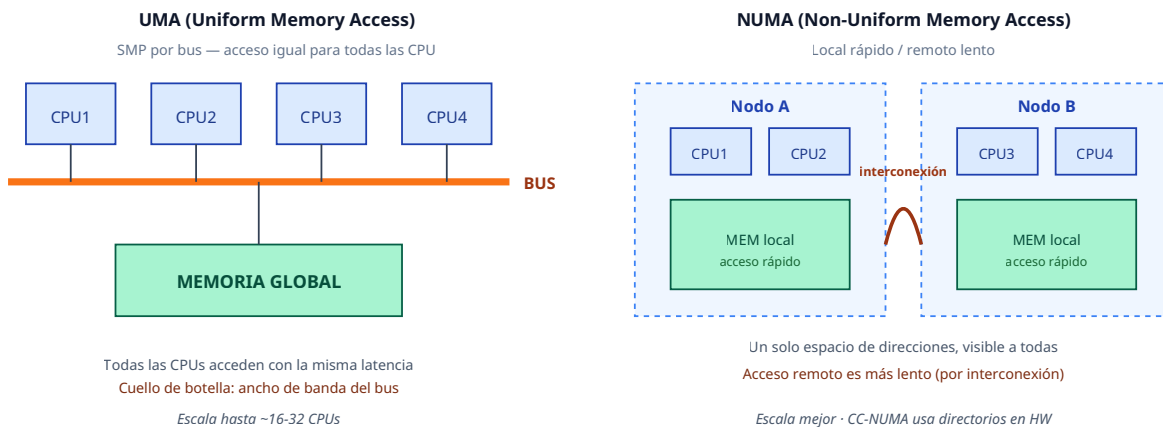


Fig. 9.1 — UMA (todas las CPU con el mismo tiempo de acceso, bus compartido) vs NUMA (memoria local rápida, remota lenta, un único espacio de direcciones).

NUMA: hay dos variantes: **NC-NUMA** (sin cache) y **CC-NUMA** (con cache; lo importante es mantener coherencia). Los CC-NUMA grandes se implementan como multiprocesadores basados en **directorios**: una base de datos en HW que indica dónde está cada línea de memoria y su estado (limpia/sucia).

Coherencia de cache: el problema central de la memoria compartida con caches. Un mismo dato puede estar cacheado en varias CPU; al escribir, la CPU manda mensaje al bus para invalidar/descartar copias limpias en otras caches, o forzar el write-back de una copia sucia antes de modificar.

Chips multinúcleo

Con más transistores por chip, las opciones son: más cache (poca mejora directa), más clock (sigue con un solo hilo, y el clock está topeado por calor), o **más cores** (comparten cache/memoria y dan paralelismo real). El software tiene que estar diseñado para aprovecharlos.

Tipos de SO multiprocesador

1. **Cada CPU con su SO** (poco usado): memoria dividida estáticamente; procesos atados a una CPU. Problemas: **desbalance** de carga, no se comparten páginas, la cache de disco puede quedar **inconsistente** entre CPUs.
2. **Maestro-Esclavo:** una sola copia del SO; **todas las syscalls van a la CPU maestra**. Resuelve balanceo y páginas, pero el maestro es **cuello de botella** con muchas CPU.
3. **SMP (Symmetric Multi-Processing):** una copia del SO, **cualquier CPU** puede ejecutarla; la syscall la corre la CPU que la invocó. Sin cuello de botella. **Problema:** varias CPU pueden estar en el kernel a la vez y competir por las mismas estructuras (elegir el mismo proceso a planificar o la misma página libre → **race condition**).

Soluciones en SMP:

- **Lock global:** todo el kernel = una sola sección crítica, una CPU a la vez. Se comporta como maestro-esclavo, mala performance.
- **Lock por estructura:** varias secciones críticas con su propio mutex. Es el enfoque más usado; mejor rendimiento pero requiere cuidado (riesgo de **deadlocks**).

Sincronización en multiprocesador

En un uniprocador basta con **deshabilitar interrupciones** para tocar tablas del kernel; en multiprocesador **NO alcanza**: otra CPU puede generar interrupciones. Se necesita **protocolo de mutex**.

TSL (Test and Set Lock): lee una palabra y la escribe en 1 en una sola operación atómica. En multiprocesador la operación **no es indivisible** si dos CPU la ejecutan al mismo tiempo: podrían leer 0 y entrar juntas. **Solución:** TSL **bloquea el bus** (requiere soporte HW). Como está esperando activamente, es un **spinlock** y genera carga (espera activa). Mejoras: variable de lock propia en cache de cada CPU, listas de espera propias por CPU, delays entre reintentos.

Planificación en multiprocesador

Se decide **qué** hilo (KLT) ejecutar y **en qué CPU**.

Hilos independientes (tiempo compartido):

- **Cola única de listos:** simple, eficiente, da **balanceo de carga**. Desventaja: **contención** sobre la estructura única.
- **Problema con espera activa:** si un hilo con spinlock pierde el quantum antes de liberar el lock, las otras CPU quedan girando ociosas. Se resuelve con un flag por proceso para no expulsarlo hasta que libere (Zahorjan 1991).
- **Afinidad de CPU:** ejecutar el hilo en la CPU donde ya corrió antes (sus datos ya están en cache). **Planificación de 2 niveles:** el hilo se asigna a una CPU al crearse; **cola por-CPU**; si una CPU queda ociosa, se reparten hilos. Minimiza la contención y el tráfico de coherencia; migrar un hilo entre CPUs implica cache misses.

Hilos que trabajan en conjunto:

- Si se planifican independientemente, no corren sincrónicos: un hilo A0 que espera A1 puede quedar en cola mientras A1 está corriendo, duplicando el tiempo total.
- **Gang scheduling (planificación por pandillas):** los hilos relacionados forman una pandilla; corren **simultáneamente** en distintas CPU e inician/terminan el intervalo juntos. Esto acelera la comunicación entre hilos que se sincronizan.

Multicomputadoras (clusters)

- CPU fuertemente acopladas, **sin memoria compartida**. PCs con interfaz de red de alto rendimiento (Ethernet 10-400 Gbps, Infiniband 120 Gbps; HBA para storage FibreChannel).
- **Topologías**: estrella, anillo, malla/mesh, hipercubo, full mesh.
- **Conmutación**: store-and-forward (buffer hasta armar paquete, latencia) vs circuitos (ruta fija, más rápida, sin control de flujo).
- **Problema típico**: copiado excesivo de paquetes entre RAM y placa. Soluciones: interfaz en espacio de usuario (sin pasar por kernel) o canales DMA / procesadores de red (lo actual).
- **Comunicación**: `send` / `receive` (con bloqueo, sin bloqueo + copia, sin bloqueo + interrupción) o **RPC** (invoca procedimiento remoto, transparente al programador).
- **Balanceo de carga**: enfoque por **grafo** (minimizar aristas/tráfico entre subgrafos asignados a distintas máquinas) o **distribuido** (los nodos reubican procesos si están sobrecargados).

Sistemas distribuidos

Tanenbaum los define como “un conjunto de computadoras independientes que el usuario ve como un único sistema coherente”. A diferencia de las multicomputadoras, tienen **menor acoplamiento**: nodos distribuidos por todo el mundo, PCs completas, con **distinto SO, HW y FS**.

Se apoyan en una capa de **middleware** por encima del SO, que aporta uniformidad, resuelve la heterogeneidad y da interfaces/servicios comunes (por ejemplo un directorio global de recursos).

Cuidado con la trampa: en un sistema distribuido los nodos **no** necesitan ejecutar el mismo SO ni tener el mismo hardware para interoperar; justamente el middleware existe para resolver esa heterogeneidad.

Conceptos clave

- **UMA vs NUMA** (uniforme vs dependiente de local/remoto), y **NC-NUMA vs CC-NUMA** (sin cache vs con cache).
- **Memoria compartida vs distribuida**: comunicación (memoria vs mensajes), acoplamiento, retardos (nseg vs µseg/mseg).
- **SMP**: definición, ventaja (sin cuello de botella) vs maestro-esclavo. Solución de sincronización: lock global vs lock por estructura.
- **Coherencia de cache**: por qué surge y cómo se resuelve (RO/RW, copias limpias/sucias, directorios en CC-NUMA).
- **TSL/spinlock**: por qué deshabilitar interrupciones no alcanza en multiprocesador; el HW debe permitir **bloquear el bus**.

- **Afinidad de CPU** y **cola por-CPU** vs cola global; **gang scheduling** para hilos que colaboran.
- Por qué se pasó a más cores (límite de calor/clock, velocidad de la luz).
- **Middleware** como diferenciador de los sistemas distribuidos.
- En SMP, migrar un hilo entre CPUs implica **cache misses** y tráfico de coherencia (por eso el SO prefiere afinidad).

10. Deadlocks

Definición

Un conjunto de procesos está en **deadlock** cuando cada uno está **bloqueado**, reteniendo recursos y **esperando** un recurso que está retenido por otro proceso del mismo conjunto. Ninguno avanza ni libera lo suyo. El caso mínimo es el “abrazo mortal”: A tiene lo que B pide y B tiene lo que A pide.

Recursos

- **Físicos** (CPU, memoria, dispositivos) / **lógicos** (archivos, registros, semáforos).
- **Apropiables / preemptibles**: se pueden quitar sin daño (memoria, CPU).
- **No apropiables**: si se quitan a mitad de uso, el proceso falla (una escritura a un CD o impresora a medias).
- Un recurso **R_j** puede tener varias **instancias** idénticas (clase de recurso).

Secuencia de uso: **Solicitar** → **Usar** → **Liberar**. Si al solicitar no se concede, el proceso espera.

Las 4 condiciones de Coffman (1971)

Deben darse las cuatro simultáneamente. Si falta una sola, no hay deadlock. Esta es la base de la prevención.

1. **Exclusión mutua**: el recurso no es compartible; solo un proceso lo usa a la vez.
2. **Retención y espera (hold and wait)**: el proceso mantiene recursos asignados mientras espera otros.
3. **No apropiación (no preemption)**: no se pueden quitar los recursos a quien los posee.
4. **Espera circular (circular wait)**: existe una lista circular de procesos donde cada uno espera un recurso del siguiente.

Las 4 condiciones de Coffman — deben darse las 4 a la vez

1. Exclusión mutua	2. Retención y espera	3. No apropiación	4. Espera circular
Recurso no compartible 1 proceso a la vez <i>"solo yo puedo tenerlo"</i>	Retiene lo que tiene y pide más <i>"tengo A, pido B"</i>	Nadie le puede quitar el recurso <i>"solo yo lo libero"</i>	Ciclo $P1 \rightarrow P2 \rightarrow \dots \rightarrow P1$ esperándose entre sí <i>"todos se esperan"</i>

Prevenición = romper al menos UNA de estas condiciones.

Fig. 10.2 — Las 4 condiciones de Coffman. Si falta una sola, no hay deadlock.

Grafo de asignación de recursos

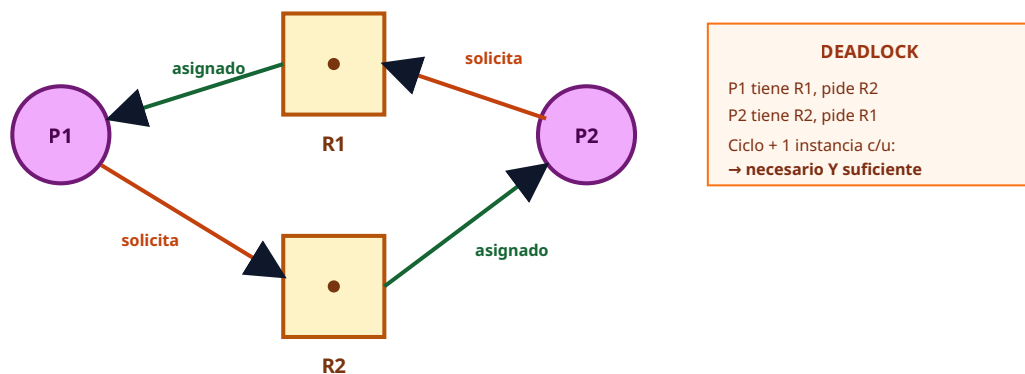
Se representan procesos como círculos (P_i), recursos como rectángulos (R_j), y con puntos internos las instancias del recurso. Las **aristas dirigidas**:

- $P_i \rightarrow R_j$: el proceso **solicita** una instancia.
- $R_j \rightarrow P_i$: el recurso está **asignado** al proceso.

Interpretación:

Estado del grafo	Resultado
Sin ciclos	No hay deadlock (seguro)
Con ciclo + 1 instancia por recurso	Sí hay deadlock (ciclo es condición necesaria y suficiente)
Con ciclo + varias instancias	Posibilidad de deadlock (necesario, NO suficiente)

Grafo de asignación de recursos con ciclo → deadlock



Con varias instancias por recurso, el ciclo es solo condición necesaria (posibilidad).

Fig. 10.1 — Grafo de asignación de recursos con abrazo mortal entre P1 y P2.

Estrategias de manejo

Estrategia	Idea	Cómo
Prevención	Que nunca se cumpla una de las 4 condiciones	Restringe la forma de solicitar recursos
Evitación (avoidance)	Asignar con cuidado según el estado del sistema	Algoritmo del Banquero
Detección y recuperación	Permitir el deadlock y luego romperlo	Grafo wait-for; abortar procesos / expropiar (selección de víctima)
Avestruz (ostrich)	Ignorar el problema	Asumir que casi nunca ocurre (lo usan UNIX/Windows en la práctica)

Prevención — cómo atacar cada condición:

- **Exclusión mutua:** hacer recursos compartibles / spooling (no siempre posible).
- **Retención y espera:** reservar TODOS los recursos al inicio, o solo pedir cuando no se tiene nada. Riesgo: starvation, baja utilización.
- **No apropiación:** virtualizar el recurso vía un demonio que encola las peticiones (spooler).
- **Espera circular:** ordenar los recursos con una función $F: R \rightarrow N$; solo se puede pedir R_j si el mayor R_i que se tiene cumple $F(R_i) < F(R_j)$.

Estado seguro vs inseguro

- **Estado seguro:** existe una **cadena segura** $\langle P_0, \dots, P_n \rangle$ con todos los procesos que pueden completar su ejecución con los recursos disponibles (en algún orden). Garantiza que **no hay deadlock**.
- **Estado inseguro:** no se puede construir esa cadena. **Posibilidad** de deadlock, no implica deadlock seguro.
- **Si hay deadlock, el estado es inseguro;** la recíproca **no vale**.

Algoritmo del Banquero (múltiples instancias)

Cada proceso declara su **máximo** de instancias que puede necesitar. El SO solo asigna recursos si el estado resultante sigue siendo **seguro**.

Estructuras:

- **disponible** (vector m): instancias libres por recurso.
- **asignacion** (matriz $n \times m$): lo que cada P_i ya tiene.

- **max** (matriz $n \times m$): máximo que **Pi** necesitará.
- **need = max - asignacion**: lo que le falta a cada proceso.

El algoritmo busca una **secuencia segura**: intenta encontrar un **Pi** cuyo **need** sea $\leq$ **disponible**; si lo encuentra, simula que **Pi** termina, libera sus recursos, y busca otro. Si al final logró que todos terminen, el estado es seguro.

Para **1 instancia por recurso** no hace falta banquero: basta el grafo de asignación y verificar que no haya ciclos.

Deadlock vs Starvation

Deadlock	Starvation
Conjunto de procesos bloqueados mutuamente , ninguno progresa	Un proceso nunca obtiene el recurso (siempre pospuesto)
Espera circular entre ellos	No requiere ciclo; suele ser por planificación / prioridades
Se rompe matando/expropiando	Se evita con políticas justas (aging, FIFO)

En la recuperación por selección de víctima, hay que evitar elegir siempre al mismo proceso para no generar starvation.

▲ TRAMPAS TÍPICAS DEL PARCIAL

- Nombrar las **4 condiciones de Coffman** y saber que las 4 deben darse simultáneamente.
- Interpretar el grafo: ciclo + una instancia por recurso = deadlock (necesario y suficiente); ciclo + varias instancias = solo posibilidad.
- Diferencia entre las 4 estrategias, y cómo prevención ataca cada condición.
- Diferencia **estado seguro vs inseguro** (inseguro $\neq$ deadlock; deadlock $\Rightarrow$ inseguro).
- Algoritmo del Banquero: reconocer las estructuras y saber construir/justificar una secuencia segura.
- **Deadlock $\neq$ starvation**: deadlock = bloqueo circular mutuo; starvation = postergación infinita, sin necesidad de ciclo.

11. Lo que más cae en el parcial

Basado en el análisis de parciales 2022, 2023 (1ra y 2da fecha), 2025 (1ra fecha) y 2026 (1ra fecha), estas son las preguntas y bloques temáticos que se repiten. Reservá los últimos días de estudio para revisar bien estos ítems.

Preguntas conceptuales que aparecieron literales o casi literales

Kernel y compilación:

- "¿Qué es el kernel? ¿Cuáles son sus funciones principales?" (2023).
- "Explique la arquitectura del kernel Linux, tipo, modularidad y portabilidad" (2025).
- "¿Qué contiene el initramfs? ¿Bajo qué condiciones puede no ser necesario?" (2023 y 2025).
- "Motivos para compilar un kernel" (2023).
- "Al compilar con `make modules_install && make install`, ¿en qué directorios se instalan el kernel y los módulos?" (2026).
- V/F: "El kernel de Linux es un SO en sentido estricto porque contiene todo lo necesario para gestionar hardware y procesos" (2026) → **Falso**.
- V/F: "En un microkernel los drivers y servidores de FS se ejecutan en modo kernel para mejorar el rendimiento" (2026) → **Falso**.

Módulos y drivers:

- "¿Qué es un módulo y qué ventaja da compilar como módulo vs built-in?" (2023 y 2025).
- V/F: "Los drivers acceden a las funcionalidades del kernel a través de system calls" (2023) → **Falso** (acceso directo por API interna).
- "¿Para qué se usa `register_chrdev()`? ¿Se puede usar en espacio de usuario o de kernel?" (2023) → **Solo en espacio de kernel**.
- "¿Qué sucede cuando se carga un módulo con `insmod`?" — multiple choice, respuesta: el módulo se carga en espacio de kernel, se inicializa con `init_module` y puede registrar interfaces (2025).

/dev y device files:

- "¿Qué tipos de archivos hay en `/dev` y qué representan?" (2023).
- Interpretación del output `brw-rw---- 1 root root 240, 0 weird`: multiple choice → **dispositivo de bloques + major 240 identifica al driver**; no son "240 bytes" ni "escribir con echo aumenta el tamaño" (2026).

System calls:

- V/F múltiple: "libc es el componente del kernel donde se definen las syscalls" → **Falso**; "las syscalls son módulos administrables con insmod/rmmod" → **Falso**; "las syscalls se ejecutan en modo privilegiado e identificadas por un número" → **Verdadero**; "cada driver implementa una syscall" → **Falso** (2023).
- V/F: "libc implementa directamente las syscalls en modo kernel sin trap ni interrupción" (2026) → **Falso**.
- V/F: "La tabla de syscalls (`syscall_64.tbl`) permite que el dispatcher identifique qué función ejecutar según el número del registro" (2026) → **Verdadero**.
- "¿Qué es una syscall y cuál es su propósito principal? ¿Cómo se implementa una nueva?" (2025).

Threads:

- "¿Quién planifica los ULT y los KLT? ¿Cómo afecta al multinúcleo?" (2025).
- "¿Qué características tendrá el proceso creado si se ejecuta `fork()` pero no `exec()`?" (2025).
- V/F: "Un hilo es la unidad básica de utilización de CPU en los SO modernos" (2026) → **Verdadero**.
- V/F: "Un ULT en M:1 puede aprovechar directamente múltiples procesadores físicos sin ningún mecanismo adicional" (2026) → **Falso**.
- V/F: "Si un proceso crea N ULTs entonces con `strace` podemos observar que invoca N veces `clone3`" (2026) → **Falso** (los ULT no llegan al kernel).

Virtualización:

- "Diferencia entre Hypervisors tipo 1 y tipo 2" (2025).
- V/F: "El SO guest tiene acceso directo y completo al HW real sin intermediación" (2026) → **Falso**.
- V/F: "En hypervisor tipo 2 el SO host se ejecuta directamente sobre el HW físico y gestiona los drivers reales" (2026) → **Verdadero**.

cgroups y namespaces:

- "¿Qué mecanismo del kernel Linux permite limitar el uso de CPU de un proceso a un 80%?" (2023) → **cgroups**.
- "Describa brevemente las funcionalidades de CGroups y Namespaces" (2025).
- "¿Por qué `mi_proceso` tiene distinto PID visto desde el contenedor y desde el host?" (2023) → **PID namespace**.

Docker:

- "¿Qué es una imagen? ¿Y un contenedor? ¿Cómo se relacionan?" (2023, 2025, 2026).

- "¿Qué características del kernel usa Docker para proveer containers?" (2023).
- "¿Qué es un Union Filesystem? ¿Cómo lo usa Docker?" (2023 y 2025).
- "¿Qué es el archivo compose y en qué se diferencia de un Dockerfile?" (2023).
- "Dos formas de persistir datos en Docker (volumes vs bind mounts)" (2023).
- V/F: "Docker usa una arquitectura cliente-servidor donde `dockerd` es el servidor responsable de crear, ejecutar y monitorear los contenedores" (2026) → **Verdadero**.
- V/F: "Al remover un archivo dentro de un Dockerfile con `RUN rm`, el archivo se elimina definitivamente de la imagen reduciendo su tamaño total" (2026) → **Falso**.

File Systems / RAID / LVM:

- "Defina inodo, bloque y extent" (2023).
- "¿Qué es un link simbólico? ¿En qué se diferencia de un hard-link? ¿Cuál/es consumen un i-nodo?" (2023 2da fecha).
- "Describe cómo BTRFS usa CoW" (2023 2da fecha).
- "Describe RAID 5 y responda cuántos discos pueden fallar sin pérdida" (2023 2da fecha).
- V/F: "RAID 1 provee striping y paridad distribuida" (2023) → **Falso**.

Seguridad:

- "¿Qué es ASLR? ¿Linux lo provee para procesos de usuario? ¿Y para el kernel?" (2025 y 2026) → **Sí para ambos, KASLR aparte**. Puede activarse/desactivarse sin recompilar (con `/proc/sys/kernel/randomize_va_space`).
- V/F: "Un sistema con muchas features tiende a ser más seguro porque ofrece más herramientas de defensa" (2026) → **Falso** (más superficie de ataque).
- V/F: "Un dominio de protección es un conjunto de pares (objeto, derecho)" (2026) → **Verdadero**.

Multiprocesadores y distribuidos:

- V/F: "En gang scheduling todos los hilos de un proceso se programan simultáneamente en distintas CPUs durante el mismo intervalo" (2026) → **Verdadero**.
- V/F: "En un sistema distribuido todos los nodos deben ejecutar el mismo SO y disponer del mismo hardware para garantizar la interoperabilidad" (2026) → **Falso**.

Bloques de contenido que casi siempre están

1. **Definición de kernel + funciones + tipo (monolítico híbrido) + portabilidad.** Aparece en 2 de las 3 primeras fechas relevadas.
2. **Módulos vs built-in + qué hace `insmod`.**
3. **Contenido y utilidad del initramfs.**
4. **Naturaleza de las syscalls y de libc.** Suele venir como multiple choice.

5. **ULT vs KLT + fork sin exec.**
6. **Imagen vs contenedor + qué usa Docker del kernel + Union FS.**
7. **cgroups (limitar CPU) + namespaces (doble PID).**
8. **ASLR (para usuario y para kernel).**
9. **File system básico (inodo/bloque/extent) + links + BTRFS + RAID 5.**
10. **Gang scheduling y sistemas distribuidos (interoperabilidad heterogénea).**

Formato del parcial

Las preguntas son de tres tipos:

- **Conceptuales cortas** (“explique X en sus palabras”).
- **Verdadero/falso con justificación breve** — pesan mucho; hay que justificar aunque parezca obvio.
- **Multiple choice** con opciones plausibles (“A y C”, “todas”, “ninguna”). Estudiá bien las opciones cercanas: el error típico es marcar la que parece “más verdadera” sin descartar las incorrectas.

ESTRATEGIA DE ÚLTIMOS DÍAS

1. **Día 1-2 (mientras lees esto en el iPad):** lectura corrida del resumen, tema por tema. No pares a memorizar todavía, dejá que el mapa mental se arme.
2. **Día 3-4:** re-lectura activa; para cada bloque, cerrá los ojos y trata de reproducir sus 5-10 puntos clave. Volvé a los “trampas típicas” al final de cada capítulo — son las que más caen.
3. **Día 5-6:** hacé de memoria los parciales viejos (2023 primera y segunda, 2025 primera, 2026). Anotá dónde te trabás y volvé al capítulo correspondiente. El 2026 tiene muchos V/F concentrados; ideal para simulacro rápido.
4. **Día 7-9:** repaso de “trampas típicas” y cheatsheet ([Infogramas_Parcial/00_INDI-CE_y_cheatsheet.md](#)). Enfocá en RAID/LVM y en las respuestas V/F contraintuitivas (paravirtualización modifica el guest; setuid usa privilegios del dueño no del que ejecuta; snapshots LVM no copian nada al crearse; etc.).
5. **Día 10 (víspera):** revisá solo esta sección 11 y las tablas comparativas. Descansá y llegá fresco.

\$ exit 0